

Mech-Mind Programming Handbook for Machine Tending

A general engineering handbook for building vision-guided load/unload cells with Mech-Eye 3D cameras, Mech-Vision, Mech-Viz and the Standard Interface.

Reference implementation: solution MechMaster, project Drill_Tap (Mech-Vision 2.1.1, Mech-Eye NANO ULTRA, Mech-Mind IPC STD).

Document	Mech-Mind Machine Tending Programming Handbook
Revision	1.0
Date	2026-08-12
Owner	Icr247
Software baseline	Mech-Vision / Mech-Viz 2.1.x (observed build 2.1.1). Where only 2.2.1 ("latest") text exists, it is marked. Legacy 1.7.x names are given for older sites.
Hardware baseline	Mech-Eye NANO ULTRA (24 VDC / 3.75 A), Mech-Mind IPC STD (IPCW-i5-16G-512G)

Contents

§	Section
---	---------

- | | |
|----|---|
| 0 | How to use this handbook |
| 1 | Machine tending as a vision problem |
| 2 | System architecture |
| 3 | Hardware and physical design |
| 4 | Bring-up sequence |
| 5 | Camera setup for machined metal |
| 6 | Hand-eye calibration |
| 7 | The vision project |
| 8 | Step reference (machine-tending subset) |
| 9 | Fixture-side and cell-state logic |
| 10 | Motion planning: Path Planning step vs Mech-Viz |
| 11 | The Output step as an interface contract |
| 12 | Robot ↔ vision communication (Standard Interface) |
| 13 | Robot-side programs |
| 14 | Error handling and recovery |
| 15 | Cycle time engineering |

§	Section
16	Changeover and multiple part numbers
17	Deployment and production
18	Commissioning and acceptance
19	Maintenance and troubleshooting
A	Status codes
B	Command quick reference and legacy names
C	Robot routine cross-reference
D	Naming and project hygiene conventions
E	Cell-specific record (fill in per installation)
F	Items to verify on your build
G	Glossary
H	Sources

0. How to use this handbook

0.1 What this is

This is a *general* programming handbook: it describes how to build **any** machine-tending application on the Mech-Mind stack — loading and unloading CNC lathes, mills, drill/tap machines, presses, deburring cells, washers — rather than documenting one finished project. The `Drill_Tap` project is used throughout as the worked reference, because it already contains the canonical pattern: **capture** → **recognise** → **gate** → **filter** → **sort** → **reorder** → **trim** → **plan** → **output**.

Read it in one of three ways:

You are	Read
Designing a new cell	§1 → §2 → §3 → §4, then use §18 as the schedule
Building the vision project	§5 → §6 → §7 → §8 → §9, with §11 as the interface contract
Writing the robot program	§12 → §13 → §14, with Appendix A/B/C on the bench
Supporting a running cell	§14 → §17 → §19, with Appendix A open

0.2 Verification legend

Every technical claim in this handbook carries one of three confidence markers. Machine tending puts a robot arm inside a machine tool; the difference between "documented" and "we think so" matters.

Marker	Meaning
[D]	Documented by Mech-Mind. Traceable to an official page (see Appendix H).
[E]	Engineering practice. Not in the manuals; standard integration practice, or

Marker	Meaning
	derived from the documented behaviour. Safe, but verify against your build.
[?]	Unverified or build-specific. Named in Appendix F. Confirm on your machine before relying on it.

0.3 Conventions

- **Poses.** Internally Mech-Vision carries object poses as **quaternions** — $[x, y, z, q_w, q_x, q_y, q_z]$, 7 components **[D]**. On the wire to a robot, poses are **always 6-DoF Euler**: X Y Z in **mm**, A B C in **degrees**, in the Euler convention of the selected robot brand **[D]**.
- **Frames.** *Camera frame* = the camera's optical frame. *Robot frame* = the robot base frame. *World frame* = the simulation origin in Mech-Viz / the path planning tool. Every pose in this handbook is annotated with its frame; unannotated poses in a real project are the number-one source of commissioning defects **[E]**.
- **Steps** are written in Title Case exactly as they appear in the Step Library — Sort 3D Poses V2, not "sort poses".
- **Commands** are Standard Interface command numbers — 101, 102, 105.
- **Vision point** = an object recognised by Mech-Vision (pose, label, dimensions, custom data). **Waypoint** = a point the robot passes through on a planned path (pose, label, motion type) **[D]**. These are different payloads fetched by different commands; conflating them causes status 1016.

1. Machine tending as a vision problem

1.1 What makes it different from bin picking

Machine tending looks like bin picking with an extra place position. It is not. Four differences drive every design decision in this handbook:

	Bin picking (to conveyor/tote)	Machine tending (to chuck/vise/nest)
Place tolerance	Loose — anywhere in the drop zone	Tight — the part must seat . A three-jaw chuck or a vise on a drill/tap machine wants sub-millimetre concentricity and a controlled orientation
Consequence of a bad pick	Part dropped, cycle retried	Part crashed into the fixture, jaws closed on air, spindle started on an unseated blank, tool broken
Cycle coupling	Robot is the only actor	Robot is a servant of the machine cycle — door open, chuck open, coolant off, air blast done, part unclamped
State	Stateless: pick next part	Stateful: is the nest occupied ? Is this a load cycle or an unload cycle?

Consequences you will see repeated later: orientation validation is mandatory, not optional (§7.5); the cell needs an explicit occupancy test (§9.2); and the robot program — not the vision project — owns the machine handshake (§2.3).

1.2 Cell archetypes

Pick the archetype first; it determines your camera mounting, your project count, and whether you need Mech-Viz.

#	Archetype	Infeed	Vision job	Typical planner
A1	Single-station load only	Bulk bin or tote of blanks	Find one pickable blank per cycle	Path Planning step
A2	Load + unload, single gripper	Bin in, finished-parts tray out	Find blank; confirm nest state	Path Planning step + robot logic
A3	Load + unload, double gripper	Layer tray / kitted pallet	Find blank; unload finished part in the same trip	Mech-Viz
A4	Multi-station / tombstone	Pallet of blanks	Find blank; select station; orientation-critical insertion	Mech-Viz
A5	Mixed part numbers	Kitted tray, several SKUs	Recognise <i>which</i> part, then locate it	Either + parameter recipes (§16)
A6	Orderly tray unload → machine	Layered blue-marked tray (as in the reference cell)	Locate every nest position, sequence them	Path Planning step + Sort 3D Poses V2

The reference Drill_Tap cell is an **A6/A2 hybrid**: parts stand in a dense grid fixture, the project recognises target object `t1`, validates orientation twice, sorts, trims, and hands the robot a planned path.

1.3 The tolerance budget

Before writing a single step, write the budget. Mech-Mind's own design guidance is the anchor:

- **±3–5 mm is enough for common projects [D]**. Most machine tending is *not* common — it is at or below ±1 mm at the fixture.
- For **±1 mm class** work, Mech-Mind directs you to deploy **Vision System Drift Auto-Correction** to counter temperature-driven error [D].
- **Camera vision-guided picking accuracy** (not metrology accuracy): NANO ULTRA-GL **±1 mm**, PRO S-GL **±0.5–±1 mm**, PRO M-GL **±1.5–±2 mm**, LSR family **±1.5–±3 mm**, DEEP-GL **±5 mm** [D].
- Do **not** quote the VDI/VDE 2634 figure (0.1 mm @ 0.6 m for NANO ULTRA) as system accuracy. That is a metrology number for the sensor alone; the classic spec error is conflating it with picking accuracy [D].

A workable budget for a chuck-loading cell [E]:

Contributor	Typical	Notes
Camera picking accuracy	± 1.0 mm	Model-dependent, above
Hand-eye calibration residual	< 2.0 mm (high-precision acceptance band)	§6.5
Robot repeatability	± 0.05 – 0.1 mm	From the robot datasheet
Gripper compliance / part slip in jaws	0.2 – 1.0 mm	Measure it; do not assume
Fixture / chuck lead-in	–(chamfer)	This is your <i>friend</i> : a 2 mm lead-in chamfer buys 2 mm
Thermal drift over a shift	0.3 – 1.0 mm	Mitigate with drift auto-correction [D]

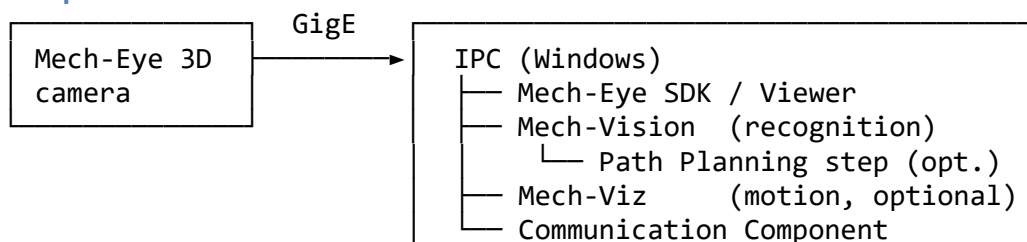
Design rule [E]: if the sum of the first four exceeds the fixture's lead-in, the cell will fail intermittently no matter how good the vision project is. Fix it mechanically — add a chamfer, a compliant gripper, a re-grip station, or a secondary positioning platform — before tuning software. Mech-Mind's own application templates route difficult loading through a "*secondary platform*" / "*positioning platform*" for exactly this reason [D].

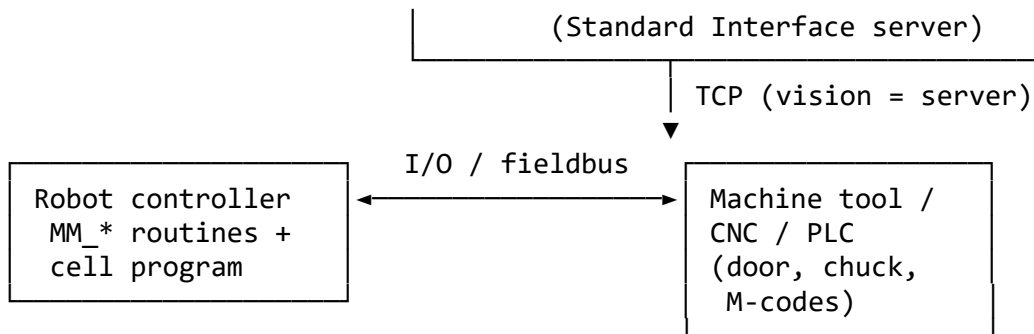
1.4 What vision must deliver, in order of importance

1. **A pose that is graspable** — not just detected. The pick point must be reachable with the actual gripper, without collision with bin walls, neighbouring parts, or the fixture.
2. **A pose whose orientation is loadable** — a blank at 40° in a bin is unloadable into a chuck even if the pose is perfect (§7.5).
3. **Correct metadata alignment** — the label, dimensions and pick-point info travelling with that pose must belong to *that* pose (§7.7). Misaligned metadata is how a robot loads the right position with the wrong gripper recipe.
4. **A deterministic answer to "nothing to pick"** — an empty result must be distinguishable from a fault (§14).
5. **Repeatable order** — for tray/pallet unloading, the sequence must be legible to an operator (§7.6).

2. System architecture

2.1 Components and who owns what





Component	Owns	Does not own
Mech-Eye camera	Raw 2D image, depth map, point cloud	Any notion of a robot
Mech-Vision	Recognition, pose processing, validation, sorting; optionally local path planning	The machine handshake; long moves
Mech-Viz (if used)	Full cycle motion logic, collision-checked paths, branch/counter/DO logic	Machine M-codes
Communication Component	The Standard Interface TCP server, protocol encode/decode, status codes	Any decision-making
Robot controller	Motion, safety, the machine handshake, error recovery, retry policy	Recognition
PLC / CNC	Door, chuck/vise, coolant, part-present sensors, cell interlocks	Poses

Architectural rule [E]: the robot program is the cell's state machine. Vision answers questions; it does not run the cell. This single rule prevents the most common architecture mistake in machine tending — burying the door-open/chuck-clamp handshake inside a Mech-Viz workflow, where it becomes invisible to the maintenance electrician at 2 a.m.

2.2 Communication modes — choose once

	Master-Control	Standard Interface	Adapter
Who sends commands	Vision system → robot	External device (robot/PLC) → vision	External device → vision
Robot-side code	<i>"No programs are needed"</i> [D]	Programs written for the robot [D]	Programs on both sides [D]
Protocols	TCP, UDP	TCP, UDP, Snap7, PROFINET, EtherNet/IP, Modbus TCP, Mitsubishi MC [D]	All of the above + HTTP, WebSocket [D]
Difficulty / flexibility	Low / Low [D]	Medium / Medium [D]	High / High [D]
Workpiece loading	✓	✓	✓

	Master-Control	Standard Interface	Adapter
supported			
Gluing supported	✓	X [D]	✓

Use Standard Interface for machine tending [E]. Reasons: the robot must own the machine handshake and safety (§2.1), the documented capability gap is gluing only [D], and the customer's maintenance team can read a robot program. Master-Control is attractive for demos and awkward in production because the vision PC becomes a motion authority. The reference cell has **Robot Communication Configuration enabled** — the toolbar toggle is on — which is exactly this mode.

2.3 Where the machine handshake lives

```

ROBOT PROGRAM (owner of the cycle)
  wait  DI[machine_cycle_complete]
  wait  DI[door_open]                ← from CNC via I/O or fieldbus
  ... unload finished part (vision optional; often a taught position)
  set   DO[chuck_open]
  ... load new blank              (vision-guided: 101 → 102/105)
  set   DO[chuck_close]
  wait  DI[chuck_clamped]
  set   DO[start_cycle]             ← M-code / cycle start to the machine
  retreat clear of the door
  set   DO[door_close]

```

Vision is called only inside the "load new blank" bracket. Everything else is deterministic robot/PLC logic [E].

3. Hardware and physical design

3.1 Camera selection

For the reference hardware, the published specification [D]:

Spec	NANO ULTRA-GL
Working distance	350M variant: 250–500 mm (object focal 350 mm) · 700M variant: 400–800 mm (object focal 700 mm). These are two SKUs, not two ranges of one camera
FOV	220 × 165 mm @ 0.25 m; 400 × 270 mm @ 0.4 m; 500 × 340 mm @ 0.5 m; 770 × 550 mm @ 0.8 m
Z repeatability (1σ)	0.045 mm @ 0.4 m; 0.1 mm @ 0.6 m
VDI/VDE 2634 Pt. II	0.1 mm @ 0.4 m and @ 0.6 m
Point cloud resolution	2400 × 1800 (4.3 MP), reducible to 1200 × 900

Spec	NANO ULTRA-GL
Capture time	0.5–0.9 s
Light source	Blue LED, 440 nm, RG2
Vision-guided picking accuracy	±1 mm
IP / temperature / humidity	IP65 / 0–45 °C / 0–85 % RH non-condensing
Power	24 VDC, 3.75 A
Size / weight	125 × 46 × 76 mm / 0.7 kg

Notes that matter in a real cell:

- **Confirm which variant you have before designing the mount.** NANO ULTRA is published in a **350M** (250–500 mm) and a **700M** (400–800 mm) variant **[D]**; the FOV and hence the usable bin footprint differ substantially. The reference unit's label reads only Model: NANO ULTRA with SN RUM70249B500E015, so read the variant from Mech-Eye Viewer or the delivery paperwork rather than the case label **[?]**.
- The **±1 mm vision-guided picking accuracy** figure comes from the **Camera Selection Guide**, not the specification sheet **[D]** — quote it as a selection-guide figure.
- Mech-Mind's model comparison lists NANO ULTRA-GL for "*metal part loading/unloading and picking*" and "*high-accuracy assembly and screw-driving*" — i.e. precision short range **[D]**.
- Mech-Mind's **machine-tending solution page recommends PRO S-GL / PRO M-GL / LSR L-GL [D]**. If your standard is NANO ULTRA, be deliberate: it is the precision/short-range choice, and FOV at 0.5 m is only ~500 × 340 mm. A 600 × 400 mm pallet will not fit in one frame — plan multiple capture positions or a different camera.
- **Mounting mode:** the model-comparison table lists NANO ULTRA-GL for **eye-in-hand [D]**; **LSR XL is ETH-only, other models support both [D]**. Confirm ETH support for your exact SKU with Mech-Mind before designing a fixed-post mount **[?]**.

3.2 Eye-to-hand vs eye-in-hand for machine tending

	Eye-to-hand (fixed post)	Eye-in-hand (on the flange)
Cycle time	Better — capture can overlap robot motion; supports "capture-and-move" [D]	Worse — robot must travel to the capture pose
FOV coverage	Fixed. Large infeed needs a taller mount and a bigger camera	Excellent — fly to any region, including <i>inside</i> the machine
Calibration	ETH process; simpler ongoing	EIH process; more sensitive to TCP accuracy
Cabling	Static	Dress pack, flex cycles, a lifetime item

	Eye-to-hand (fixed post)	Eye-in-hand (on the flange)
Machine-side inspection	Can't see into the machine	Can — useful for nest occupancy and chip detection
Collision risk	None (camera out of the work zone)	Camera becomes part of the tool envelope; model it in collision detection

Default for machine tending [E]: eye-to-hand over the infeed, because cycle time is the commercial metric and the fixed post keeps the camera out of coolant and chips. Choose eye-in-hand when the cell must *look inside* the machine (nest occupancy, chip check) or when the infeed area exceeds one FOV.

3.3 IPC setup

The Mech-Mind IPC ships configured [D]; if you build your own, or reimage, these are the documented requirements:

Item	Requirement
CPU	Must support the AVX2 instruction set [D]
GPU driver	Mech-Vision 1.7.0 / Mech-DLK 2.3 require NVIDIA 472.50+ [D]
Region & language	IPC region and language must match, or logs render garbled [D]
Firewall	Disable Windows Defender Firewall on the adapters facing camera / robot / PLC, across Domain, Private and Public profiles [D]
Windows Update	Disable both Windows Update and Windows Update Medic Service — the Medic service re-enables Update by itself [D]
License dongle	Plug full license dongles in before installing software ≥ 1.6.0 [D]
Non-deep-learning speed	Depends on CPU multi-core performance, not the GPU [D]

IPC STD hardware facts worth having on the wiring drawing [D]:

- NIC ports are labelled **CAMERA 1 / CAMERA 2 / CAMERA 3-ROBOT / ROBOT-PLC**.
- Speed LED: off = 100 Mbps, orange = 1 Gbps, green = 2.5 Gbps. **Camera links must not be at 100 Mbps** — normal camera transmission is 700–800 Mbps [D], so an orange/green LED is a commissioning check.
- **PoE is not supported** — the camera needs its own 24 VDC.
- Digital I/O connector: DIN0–DIN5, DOUT0–DOUT5, +5VS, GND on pins 1 and 8.
- DB9 serial COM1–COM4 (RS232) and COM5–COM6 (RS232/422/485).
- **Remote power connector:** a short-circuit signal on the 2-pin terminal triggers a clean shutdown — wire it to a PLC output so the cell can power the IPC down gracefully on plant shutdown.
- Start-up beep codes are in the IPC manual's Appendix A; a cell that "won't boot" is often a memory seating fault with a documented beep pattern.

3.4 Network plan

Link	Recommendation
Camera ↔ IPC	Dedicated NIC, gigabit or better, its own subnet. Never share with the plant LAN [E]
IPC ↔ robot	Dedicated NIC. IPC and controller in the same subnet , different addresses [D] (docs example: 192.168.100.169 / .170, mask 255.255.255.0)
IPC ↔ PLC	Separate NIC where available (the STD has ROBOT-PLC)
Plant LAN	Ideally not connected. If required for backups, treat as untrusted and keep Windows Update disabled [D]

FANUC-specific: if both Port 1 (CD38A) and Port 2 (CD38B) are enabled, *their IP addresses must be on different subnets* [D].

Port number. There is no single hard default [D]:

- ABB / KUKA / YASKAWA setup pages recommend **50000 or above** [D].
- Official FANUC examples use **30000**, and on **FANUC V10 the port range is 1–32767** — so 50000 is invalid there [D].
- TCP supports concurrent processing on a maximum of **four ports**; if multiple ports are used, modify the example program so global variables are not shared between ports [D].

3.5 The environment next to a machine tool

Machine tending puts a 3D camera in one of the worst optical environments in a plant. Design against it [E], with the documented levers noted:

Hazard	Mitigation
Coolant mist, oil film on the lens	Air-purged enclosure or a sacrificial window; scheduled lens cleaning (the docs list regular lens cleaning as a design constraint [D])
Chips on the infeed / in the nest	Air blast before capture; nest occupancy check (§9.2)
CNC work lights, strobing	Anti-Flicker Mode suppresses depth fluctuation from artificial lighting [D]
Ambient sunlight / high ambient light	LSR (laser) models have higher ambient-light immunity [D]
Vibration from the machine, VFD EMI	Rigid, decoupled camera post; separate cable routing; the docs call out vibration tolerance and EMI as design constraints [D]
Temperature swing over a shift	Drift Auto-Correction for ±1 mm class work [D]; camera rated 0–45 °C [D]
Camera post knocked by a forklift or an operator	Mechanical guard, and a daily calibration check in the shift start-up routine (§19.1) [E]

3.6 Safety

Out of scope for the software, but state it in every project file [E]: vision does not make a cell safe. The robot's safety-rated stop, the light curtain or fence interlock, the machine's door interlock, and the chuck-clamped confirmation are all safety-rated I/O owned by the robot/PLC. A vision result must never be a condition for a safe state — only for a motion target.

4. Bring-up sequence

Mech-Mind's own tutorial spine is the right order, and it is worth following literally because each phase's output is the next phase's input **[D]**:

Vision Solution Design → Vision System Hardware Setup → Robot Communication Setup → Hand-Eye Calibration → Workpiece Locating → Pick and Place

Expanded for machine tending **[E]**:

Phase	Output	Gate before proceeding
0. Concept	Archetype (§1.2), tolerance budget (§1.3), camera + mount choice	Budget closes mechanically
1. Hardware	Camera mounted, IPC configured, networks up, license dongle in	Camera visible in Mech-Eye Viewer; link at 1 Gbps+
2. Image quality	Capture parameter group tuned for the actual part finish	Point cloud complete on the worst-case part (§5.5)
3. Communication	Robot Communication Configuration applied, MM_* programs loaded, comms test passes	MM_COMTEST (or brand equivalent) returns a good status
4. Calibration	Hand-eye calibration saved, error within band	Error < 2 mm at 100 % for high-precision work (§6.5)
5. Target object	Point cloud model + pick points + collision model in the target object editor	Recognition succeeds on 20 consecutive frames, worst-case presentation
6. Vision project	Full pipeline (§7)	Correct pose, correct order, correct empty-result behaviour
7. Motion	Path Planning workflow or Mech-Viz project, collision models real	Simulated cycle collision-free at production speed
8. Robot program	Full cycle incl. machine handshake and error handling (§13, §14)	Dry cycle without a part; then with one part
9. Commissioning	Acceptance protocol signed (§18)	Soak test passed
10. Handover	Backups, documentation, operator training, spares	§17.5

Do not skip phase 2 or 4. A poorly exposed point cloud and a 4 mm calibration residual both present as "the recognition is unreliable", and both are invisible in the step graph.

5. Camera setup for machined metal

Machine-tending parts are the hard case: turned steel, ground shafts, black-oxide drill blanks, aluminium with a fresh cut face. All of them are specular; some are both dark and specular in the same frame.

5.1 Tune in Mech-Eye Viewer, use the group in Mech-Vision

Parameters live in a **configuration parameter group** in Mech-Eye Viewer; the Capture Images from Camera step then selects that group **[D]**. Never tune inside the vision project — tune in Viewer, save the group with a meaningful name (t1_blank_reflective), and select it in the step. That way the same group can be reused by other projects and audited **[E]**.

5.2 3D exposure — the documented method

Single-material part **[D]**:

1. Set *Exposure Multiplier* = 1, *Flash* exposure mode, *Fast* acquisition.
2. Capture once and judge the **depth-source 2D image**: too dark → increase *Exposure Time*; too bright → decrease.
3. Missing depth-map data indicates **under**exposure.

Mixed or shiny surfaces — the normal machine-tending case **[D]**:

1. Find the exposure time that images the **brightest/most reflective** region correctly.
2. Find the exposure time that images the **darkest** region correctly.
3. Set *Exposure Multiplier* = 2 and enter both values in *Exposure Time* and *Exposure Time 2*.
4. If coverage is still incomplete, add a third exposure and set the multiplier to 3.

Cycle-time alternative **[D]:** *Fringe Coding Mode* = **Reflective** gives single-pass acquisition for mixed/shiny surfaces, and is available on DEEP-GL, LSR, **NANO ULTRA-GL** and PRO. On a reflective-part cell this preserves takt time versus stacking two or three exposures — try it before you accept a 3× capture penalty.

5.3 Other levers for reflective parts

Parameter	Setting for shiny metal	Source
Light Brightness (LED projector)	Low for reflective objects (High for dark, Normal for regular)	[D]
Laser Power (laser models)	Lower intensity for highly reflective objects; higher for dark	[D]
Fringe Coding Mode	Reflective (mixed/shiny); otherwise Fast vs	[D]

Parameter	Setting for shiny metal	Source
	Accurate trade-off	
Anti-Flicker Mode	On, near CNC work lights	[D]
Gain	Use sparingly — raises brightness but adds noise	[D]

Per-model parameter defaults (Gap Filling, Fringe Contrast Threshold, Outlier Removal, Surface Smoothing, Distortion Correction, Anti-Blur) live in the camera manual's per-model *Parameter Reference Guide* chapter; pull exact values from there for your SKU [?].

5.4 Depth range and 3D ROI — also a cycle-time lever

Three parameters need Mech-Eye Viewer's visual tools and several rounds of fine tuning [D]:

1. **Auto-Exposure ROI** (2D parameters, when exposure mode = Auto).
2. **Depth Range** — upper and lower limit.
3. **3D ROI** — x, y, width, height.

Trim both to the actual working volume. It removes bin walls and machine structure from the cloud *and* cuts data volume and processing time — the documented capture-side cycle-time levers are "minimise exposure time, update firmware, **trim ROI and depth range**, gigabit network" [D].

Machine tending caution [E]: leave 20–30 mm of margin around the bin/pallet footprint for placement variation, and re-check the depth range when the tray is *empty* — a range tuned on a full tray can clip the bottom layer.

5.5 Image-quality acceptance test

Before moving on, prove the capture [E]:

- ☐ Worst-case part (darkest, shiniest, most tilted) has a complete point cloud on the graspable surface.
- ☐ Depth holes, if any, are away from the pick point and the matching features.
- ☐ Point cloud fluctuation across 10 static captures ≤ 3 mm (the documented acceptance limit for calibration-time fluctuation, a reasonable bar here too) [D].
- ☐ Full tray and near-empty tray both image acceptably.
- ☐ Capture time recorded, and it fits the takt budget (§15).
- ☐ Parameter group saved with a versioned name and backed up.

6. Hand-eye calibration

6.1 Choose the process

The calibration guide branches on four axes [D]: **robot type** (6-axis / 4-axis SCARA & palletizer / 5-axis / gantry), **communication mode** (Standard Interface or Master-

Control), **camera mounting mode** (ETH/EIH), and **data-collection method** (Automatic / Manual TCP-touch / Manual random board poses).

For a typical machine-tending cell: 6-axis, Standard Interface, eye-to-hand, **Automatic**.

6.2 Automatic ETH calibration — documented workflow

1. **Prerequisites [D]**: full system installed; calibration board on a robot-specific bracket at the flange; robot parked at the **bottom centre of the camera FOV**; 2D/3D parameters tuned in Mech-Eye Viewer so the board images cleanly; robot communication configured.
2. Mech-Vision → **Camera Calibration** → *New calibration* → *Hand-eye calibration for listed robot* → robot model → **Eye to hand** → **Automatic** → **Standard Interface**: set protocol and host IP, start the interface service, and run the auto-calibration program on the teach pendant **[D]**.
3. Select board type **Standard**, matching the model on the board's nameplate; place the board at the centre of the FOV (red rectangle) **[D]**.
4. **Check intrinsic parameters** (*Start checking*). If circle detection is poor, draw aid circles or hand-tune the detection thresholds **[D]**.
5. Configure the **pyramid calibration path**: *Height span*, *Path type = ToHand*, *Num of layers*, *Bottom/Top-layer dimensions X/Y*, *Motion grid cols and rows per layer*, *Rotation angle* **[D]**.
6. Tick **Save images** and run *Auto move robot along path and capture images* **[D]**.
7. Calculate extrinsics, inspect the error, save **[D]**.

The robot side of this handshake is **command 701**: the robot sends 701, calibration status, flange pose, joint positions where sent-status 0 = "initiate calibration", 1 = "reached previous point", 2 = "failed to reach"; the reply carries the next calibration point and a returned status of 0 = in progress, 1 = finished **[D]**. Each brand provides an MM_CALIB routine and a ready-made auto-calibration program (§13).

6.3 Manual calibration (TCP touch) — when automatic is impossible

Needed when the robot cannot carry the board, or the cell geometry blocks the pyramid path **[E]**. Documented essentials **[D]**:

- A sharp tip on the flange, an undamaged board with clear circles, board placed at the **centre of the workobject plane**.
- Wizard: *New calibration* → *Hand-eye calibration for custom robot* + Euler-angle convention → robot type → **Eye to hand** → **TCP touch** → connect camera → mount/place board → intrinsic check → **Set TCP in flange frame**.
- **At least three points not in a line** must be touched to compute extrinsics.
- Touch the cross centre of each point, read the flange pose from the pendant, type it in; then capture images with *Save images* on.
- Use *Calculate rotation and translation separately*; review the error in the point cloud viewer.

6.4 Calibration boards

Family	Models (circle spacing)	Grid	Notes
BDB	BDB-5 (20 mm), BDB-6 (35 mm), BDB-7 (50 mm)	5×4	V2/V3 camera generations [D]
CGB	CGB-020 (20 mm), CGB-035 (35 mm), CGB-050 (50 mm)	5×4	V4 cameras — current [D]
OCB	OCB-005 (5 mm), OCB-010 (10 mm), OCB-020 (20 mm)	9×11	EIH only — no mounting holes, cannot be bolted to an end effector [D]

Boards serve two purposes: the **intrinsic parameter check** and the **hand-eye calibration** itself **[D]**. For NANO ULTRA at 250–800 mm expect a **CGB-020 or CGB-035** class board — confirm against the *Calibration Board Selection* chapter before ordering **[?]**.

6.5 Acceptance — the numbers that matter

Published acceptance thresholds, all points must pass **[D]**:

Project class	Threshold
Standard projects	error < 3 mm (100 %)
High-precision projects	error < 2 mm (100 %)
(De)palletizing	error < 5 mm (100 %)

Machine tending into a chuck or vise is a **high-precision project** — hold the 2 mm band **[E]**.

Checks **[D]**:

- **Point Cloud Viewer error map** — darker = larger error; number keys **0–9** isolate error bands in **0.5 mm** increments; quote the value at the **100 % percentile**.
- **ETH**: the robot's point cloud should roughly coincide with the robot model. **EIH**: the board's point cloud must stay stable across poses.
- **Euler-angle consistency**: angles should fluctuate < **1°** along the pyramid path. More than that suggests the robot has **lost its zero position** — stop and fix the robot, not the calibration.
- **Intrinsic check**: watch "Error of measured average calibration circle interval"; out-of-spec points turn yellow.
- **Point cloud fluctuation: max acceptable 3 mm**. Exceeding it at several points means an exposure problem or mechanical instability — fix before recalculating.
- Remedies: optimise exposure, clamp the board rigidly, reduce robot speed, recompute compensation parameters.

6.6 How many poses

- Manual/TCP-touch: **≥ 3 non-collinear points [D]**.
- **More is not better**: "*Too many calibration points may introduce abnormal points, leading to an increase in the overall error ratio.*" Recommended grid by focal

distance: **2×2 for 300–2000 mm; 3×3 for 2000–3500 mm [D]**. A NANO ULTRA cell sits in the first band → **2×2 per layer**.

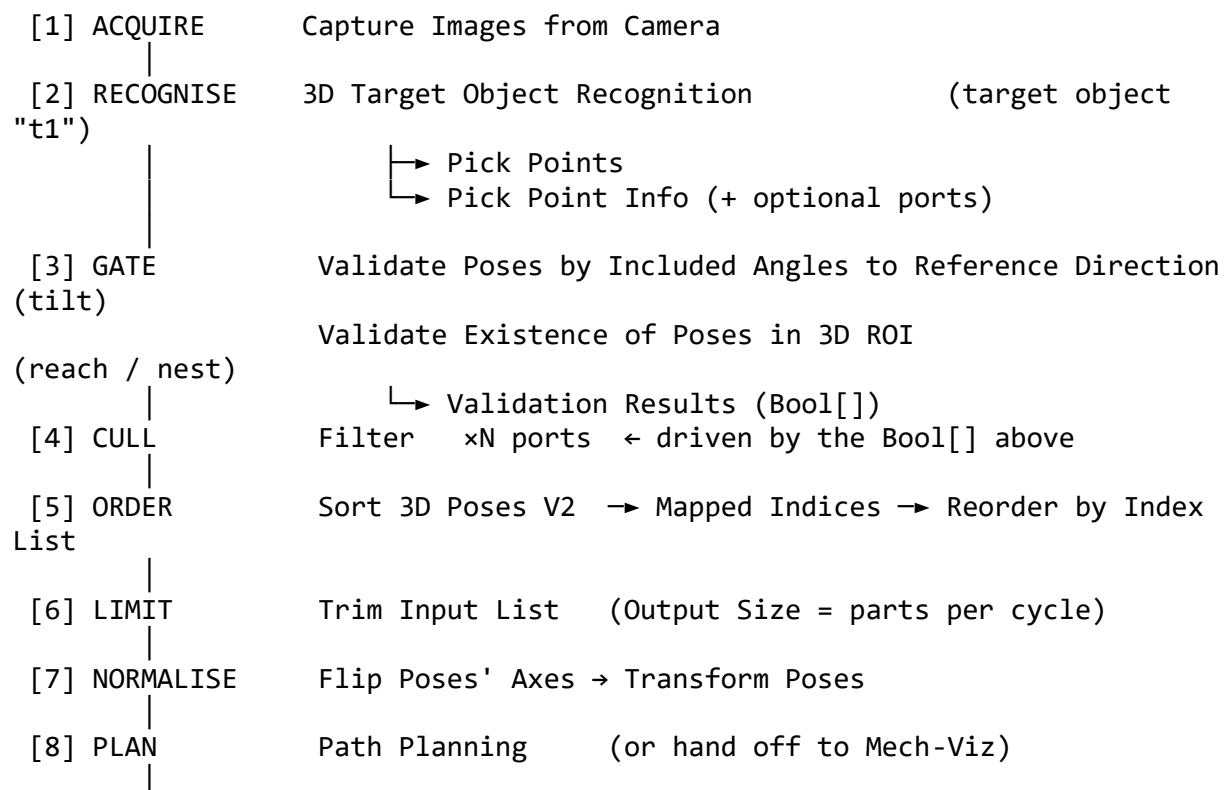
6.7 Drift, camera swaps, and when to recalibrate

- Mech-Mind ships **Vision System Drift Auto-Correction** (with ETH and EIH deployment guides and a calibration-sphere hardware guide) specifically for temperature-driven error, and recommends it for ± 1 mm class accuracy **[D]**.
 - The **Production Interface can monitor accuracy drift in production [D]** — enable it; it turns a slow failure into a visible trend.
 - A **Quick Camera Replacement** feature exists for swapping a camera without a full recalibration **[D]**; its prerequisites are not documented on the pages reachable for this handbook **[?]**.
 - **There is no official statement requiring recalibration after the camera is bumped [?]**. Treat "camera or camera post moved, robot repaired, or robot zeroed → redo hand-eye calibration" as mandatory engineering practice **[E]**, and put a witness mark on the camera bracket so a bump is *visible* **[E]**.
-

7. The vision project

7.1 The canonical machine-tending pipeline

Every machine-tending vision project reduces to seven functional stages. The reference Drill_Tap project implements all of them.



[9] EMIT	Output	(the interface contract, §11)
Parallel: output)	Show Point Clouds and Poses	(diagnostic tap, no data)
	Trigger Control by Flag	(per-cycle branching, §9)
	Easy Create Poses	(constant reference poses)

Stages 3–6 are the ones that turn "detected" into "loadable", and they are where most projects are under-built.

7.2 The reference project, step by step

Read off the Drill_Tap step graph (Mech-Vision 2.1.1, 37 steps, ~1.50 s execution):

#	Step (as named in the project)	Role in the pipeline	Handbook §
1	Capture Images from Camera (1)	Acquire depth map + colour image	§8.1
2	3D Target Object Recognition (1) — target object t1, Config wizard	Recognise blanks, emit pick points	§8.2
3	Easy Create Poses (1) / (2)	Constant reference poses for the two angle gates	§8.3
4	Validate Poses by Included Angles to Reference Direction (1)	First tilt gate (pre-normalisation)	§8.4
5	Validate Existence of Poses in 3D ROI (1)	Reach / region gate	§8.5
6	Sort 3D Poses V2 (1) / (2)	Two-key ordering (e.g. layer then row)	§8.6
7	Filter (1)–(4)	Cull all parallel lists using the Bool[] verdicts	§8.7
8	Reorder by Index List (1) / (2)	Keep metadata aligned with the sorted poses	§8.8
9	Trim Input List (1) / (2)	Keep only the parts this cycle will use	§8.9
10	Validate Poses by Included Angles to Reference Direction (2)	Second angle gate, after normalisation	§8.4
11	Trigger Control by Flag (1)	Per-cycle branch	§8.10
12	Flip Poses' Axes (1)	Force the approach axis to point into the part	§8.11
13	Transform Poses (1)	Camera → robot frame (or fixture-relative)	§8.12
14	Path Planning (1) — Config wizard	Local collision-free	§8.13

#	Step (as named in the project)	Role in the pipeline	Handbook §
		approach/pick/depart	
15	Output (1)	Emit to the Standard Interface	§8.14 / §11
—	Show Point Clouds and Poses (1)	Diagnostic visualisation	§8.15

Why the double gate (steps 4 and 10) is correct [E]. Gate 1 removes grossly mis-oriented detections before any pose surgery, which keeps the downstream lists small and the sort meaningful. Gate 2 runs after `Flip Poses' Axes` normalises the approach axis, so it measures the tilt the *gripper will actually see*. A single gate placed before the flip will reject good parts whose Z happens to point up (they read $\sim 170^\circ$ off); a single gate placed after the flip will let through parts that were mis-detected rather than merely flipped.

Why the paired sorts and reorders [E]. `Sort 3D Poses V2` emits `Sorted Poses` and `Mapped Indices`. Two sorts in series give a compound key — for a layered tray, sort by Z (layer) then S-shape on plane (row/column within the layer). Each sort's indices must be applied to every parallel list via `Reorder by Index List`, or the labels stop belonging to the poses.

7.3 Project hygiene

Conventions that pay for themselves on the second visit [E]:

- **Name every step instance** for its job, not its type: `Gate_Tilt_PreFlip`, `Sort_Layer_Z`, `Trim_OnePerCycle`. The default (1), (2) suffixes are how a 37-step project becomes unmaintainable.
- **Comment the frame** on every step that touches poses. Write "camera frame in, robot frame out" on `Transform Poses`.
- **Keep a Show Point Clouds and Poses tap** at the end of the pose chain with its eye icon off. It costs nothing in production and saves an hour when a part mis-picks [D] (the step has no data output, so it cannot affect the result).
- **One Output step per project** — that is a hard limit [D]. Multi-machine cells are multiple projects in one solution.
- **Version the solution folder** before every change (§17.4).
- **Record the step count and execution time** of a known-good build (`Drill_Tap`: 37 steps, 1.50 s). It is the cheapest regression test you have [E].

7.4 Recognition: target object, model, pick points

3D Target Object Recognition integrates point-cloud preprocessing, deep learning and removal of overlapped objects, and its documented usage scenario is **workpiece loading** — separate arrangements, orderly single-layer, orderly multi-layer, and random stacking [D]. That is machine tending, exactly.

The **target object editor** consolidates point-cloud model generation/editing, symmetry settings, object centre point, **pick points**, and the **collision model**, through six guided workflows [D]. Key concepts [D]:

- A **point cloud model** is a predefined cloud reflecting the object's shape and features, matched against the scene cloud. Requirements: enough evenly distributed points (speed), typical features present (accuracy), no interfering points (stability).
- A **pick point** is a grippable position defined **in the object reference frame**. An object may have several, inside, on, or near the cloud.
- Models can be generated from **STL CAD** by setting units and sampling the exterior surface [D].
- **Compatibility**: target objects configured with 2.1.2 cannot be used with software earlier than 2.1.2 [D]. Pin your version.
- In the *Target object selection and recognition* view the visualisation shows **object centre points**; **pick points** appear only in the *General settings* view. Close the tool before an external service triggers the project, or results will not refresh [D].

Machine-tending practice [E]:

- Model the blank as it actually arrives — with the flash, the sprue, the sawn face. A pristine CAD model matches badly against a real casting.
- Define pick points that are **chuck-compatible**: grip on a surface that will not be the clamping surface, and leave the datum face clear.
- Give rotationally symmetric parts **several pick points** (clock positions) so the planner has options; then let symmetry settings and pick-point filtering choose.
- Enable **Target object confidences** and **Overlap ratios** output ports so sorting can prefer clean, unoccluded, top-layer parts [D].
- Enable **Point clouds of matched target objects** when you want post-pick error proofing (wrong part / short part) before the door closes [D].

Confidence. There is no parameter literally named "Target Object Confidence" [?]. What exists: the *Target object confidences* **output port**, and the underlying **3D Matching → Confidence Settings** — *Result Verification Degree* (Low / Standard / High / Ultra-high, default Standard) and **Confidence Threshold** (default **0.3000**), where confidence is defined as *the coincidence ratio between the point cloud model and the scene point cloud* [D]. Raise the threshold when you get false recognitions; lower it when you get false negatives. Deep-learning assist has its own separate confidence threshold, supports **instance segmentation and object detection** only, runs at **batch size 1**, and offers *Dilation / Kernel size* to stop undersized masks clipping edge cloud [D].

The **optional output ports** are added in the wizard under *General settings* → *Configure output port* [D]. Note that the port names visible on a step block in a given build may not match the documented checkbox names; the documented set is: *Port(s) related to pick point*, *Port(s) related to object centre point*, *Pick point numeric labels*, *Target object confidences*, *Original point cloud acquired by camera*, *Preprocessed point cloud*, *Point clouds of matched target objects*, *Overlap ratios*, *Deep learning visualization result* (which requires

clicking **Save**, or the port is not added) **[D]**. Names such as "Pick Point IDs", "Colored Point Cloud", "Merged Point Cloud", "Object Point Clouds" seen on step blocks map onto these but are **not documented under those names [?]** — record the mapping for your build in your project notes.

7.5 The orientation gate — why machine tending needs it

A blank lying at 35° in a bin may be recognised perfectly and still be **unloadable** into a three-jaw chuck, and a tilted approach risks a gripper-to-bin-wall collision.

Configure Validate Poses by Included Angles to Reference Direction **[D]**:

Parameter	Machine-tending setting
<i>Axis to Be Specified</i>	Z — the pick approach axis
<i>Reference Direction</i>	Positive/Negative Z of the world frame, per your convention (default Positive Z)
<i>Reference Direction Source</i>	Pose (fed from Easy Create Poses) or Vector3d
Max Angle Difference	The tilt the gripper and the fixture genuinely tolerate — commonly 15–30° for a two-finger or magnetic gripper feeding a lathe (default is 90°, which is far too permissive) [E]

Wire Validation Results into a **Filter** so the *whole* parallel data set is culled consistently, not only the pose list. In 2.2.x the step also emits Angles (Number[], degrees) — log it as a process variable to watch tilt distribution drift as the tray empties **[D]/[E]**.

Related steps, so you pick the right one **[D]**:

Step	Use
Validate Poses by Included Angles to Reference Direction	The canonical angle gate
Calc Included Angles between Specified Axes of Poses	Compute an angle between two pose lists (Advanced Step)
Validate Pose Angle and Position of Target Objects	Validates angular and positional deviation vs a reference — " <i>scenarios requiring strict control over the position and orientation of target objects</i> ", i.e. assembly-grade insertion
Adjust Poses V2 → Filter Poses by Predefined Options → Filter by angle	The same function inside the pose-adjustment tool
Rotate Axis to Minimize Included Angle to Reference Direction	Rotation, not validation — do not confuse it

7.6 Ordering: sorting *is* the picking strategy

Sort 3D Poses V2 offers eleven rules [D]. The ones that matter here:

Rule	Machine-tending use
S shape on plane / Z shape on plane	Orderly single- or multi-layer trays and pallets. Documented use: <i>"unloading neatly arranged workpieces or depalletizing"</i> . Parameters: <i>Row direction</i> (default +X), <i>Column direction</i> (default +Y), Row Interval (default 100 mm)
Z value of pose	Random bins — work top-down
Pose confidence values	Attempt the cleanest, least-occluded part first; pairs with <i>Overlap ratios</i>
Distance from pose to reference pose (or on XOY)	Minimise travel from the current position; pairs with Easy Create Poses as the reference
Projection of pose in reference direction	Unload along a defined axis, e.g. front-to-back in a fixture

Align the row direction with the tray's long axis so the order is legible to an operator, and so robot travel stays monotonic and takt time stable [E].

7.7 Metadata alignment — the defect that hides longest

Sort 3D Poses V2 emits Mapped Indices; Reorder by Index List applies them [D]. Set Port Number to the number of parallel lists (up to 15) and drive them all from one Indices input. 2.2.1 states the intent explicitly: *"suitable for scenarios where multiple relevant data (such as pick points, pick point information, and numeric labels of pick points) need to be sorted simultaneously"* [D].

The same applies to Filter (multi-port) and Trim Input List (multi-port). **Rule [E]: any step that changes the order or length of the pose list must change every parallel list identically, in the same step instance.** Sorting poses without reordering their metadata is how a robot picks part A using part B's label — and it will pass every bench test, because on the bench the lists happen to be in the same order.

7.8 Normalisation before planning

Flip Poses' Axes forces the approach axis into the part: *Axis to Flip = Z*, *Direction Type* per your convention, *Reference Axis to Rotate Around = X* (it must differ from the axis being flipped) [D]. Matching returns a pose consistent with the point-cloud model, so a rotationally ambiguous blank — a symmetric flange, a shaft, a plate — can come back with Z pointing *out of the bin*, which the robot cannot approach. Normalising here reduces "unreachable pose" and near-singularity planning failures, and it is what makes the second angle gate meaningful [E].

Transform Poses then changes frame [D]:

Transformation Type	Use
CameraToRobot (default)	The standard frame change every project needs
RobotToCamera	Inverse
AllWithFirst	The first reference pose transforms all original poses — express blanks in a <i>measured fixture frame</i>
FirstWithAll	All reference poses applied to the first original pose
UseCorrespondenceInput	Pairwise — per-station origins on a multi-station tombstone

Always confirm what the *next* step expects: Validate Existence of Poses in 3D ROI has an explicit Input Poses Coordinate Type, and Path Planning/Output have a Point Cloud in Camera Frame checkbox for the same reason **[D]**.

7.9 Limiting the result

A machine tool consumes one part per cycle, and planning cost grows with the number of vision points evaluated. Put Trim Input List after sorting with Output Size = 1 for single load/unload, or = the number of parts a multi-gripper carries [D]/[E]. This also reconciles with the protocol: command 101 carries an expected number of vision points, and trimming keeps Mech-Vision's answer inside what the robot program is written to consume [D].

8. Step reference (machine-tending subset)

Version note: categories and some names **changed between 2.1.x and 2.2.x [D]**. Pin your handbook to one version and say which. This section is written for **2.1.x**, with 2.2.x deltas marked **Δ**.

Step	Category 2.1.2	Category 2.2.1
Capture Images from Camera	Capture	Data Acquisition
3D Target Object Recognition	Recognition	Recognition
Easy Create Poses	Data Processing	Tools
Validate Poses by Included Angles to Reference Direction	Evaluation	Evaluation
Validate Existence of Poses in 3D ROI	Evaluation	Evaluation
Sort 3D Poses V2	Pose Processing	Pose Processing
Filter	Evaluation	Evaluation, renamed Filter Data by Boolean Value
Reorder by Index List	Data Processing	Data Processing
Trim Input List	Data	Data Processing

Step	Category 2.1.2	Category 2.2.1
	Processing	
Trigger Control by Flag	Evaluation	Tools
Flip Poses' Axes	Pose Processing	Pose Processing
Transform Poses	Pose Processing	Pose Processing
Path Planning	Path Planning	Path Planning
Output	Transmission	Transmission
Show Point Clouds and Poses	Pose Processing	Pose Processing

All of the above **[D]**. Note the common misfiling: Filter is under **Evaluation**, not Data Processing.

8.1 Capture Images from Camera

Outputs [D]: Camera Depth Map (Image/Depth), Camera Color Image (Image/Color), Point Cloud (PointCloud/XYZ), Colored Point Cloud (PointCloud/XYZRGB). A Color Image Path port also exists (implied by *Image Name Type*).

Key parameters **[D]**:

Parameter	Machine-tending setting
Virtual Mode	Off in production; on for offline reproduction of a mis-pick from saved data
Camera ID / Select camera	The production camera
Calibration Parameter Group	Must be the group produced by the current hand-eye calibration, or every downstream frame transform is wrong
Configuration Parameter Group	The Mech-Eye Viewer group tuned in §5 (e.g. reflective)
Timeout	Default 10000 ms — covers connect and capture
Num of Reconnection Attempts / Max Capture Attempts	Default 3 each (3 recommended). This pair is the anti-nuisance-fault setting: a machine cycle should not hard-fault on one dropped frame
2D Image Type / Rectify to Depth Map	LSR and DEEP-GL only; leave default otherwise
Virtual mode: Data Path / Play Back Mode	Folder of rgb_image_*.jpg + depth_image_*.png + intrinsics; Sequential / Repeat one / Repeat all / Random. MRAW files cannot be read

The textured point cloud from this step is by default the project's **scene point cloud**, so images are still captured even if the step's cloud output is unconnected **[D]**.

8.2 3D Target Object Recognition

Documented ports [D]: in — Camera Depth Map, Camera Color Image; out — **Pick Points** (Pose[]), **Pick Point Info** (JsonValue). Optional ports are added in the wizard (§7.4).

Step parameters [D]: *Config wizard* (also a button on the block), **Select Target Object** (the drop-down showing t1 in the reference project), and the execution flag *Trigger Control Flow Given No Output* — when selected the control flow still fires with empty output, and the step can still output the scene point cloud; *Trigger Control Flow Given Output* always takes effect.

The tool has three processes **[D]**: point cloud preprocessing (valid recognition region, edge detection, filtering) → target object selection and recognition (optional deep-learning assist, matching parameters) → general settings (output ports only).

8.3 Easy Create Poses

No input; output Pose (Pose[]) **[D]** $\triangle$ 2.2.1 port table. Parameter **Poses**: each pose is 7 components [x, y, z, qw, qx, qy, qz], default [0,0,0,1,0,0,0], multiple poses separated by semicolons **[D]**.

Use for any constant, unsensed pose: the reference pose for an angle gate, a taught chuck-face or vise-jaw reference, the reference pose for distance sorting, or a hard-coded fallback drop pose during bring-up **[E]**. Always record which frame the literal is expressed in — this is the single most common source of confusion in load/unload projects **[E]**.

8.4 Validate Poses by Included Angles to Reference Direction

Ports **[D]**: in Poses (Pose[]), optional Reference Poses (Pose[]), optional Reference Directions (Vector3D[]); out Validation Results (Bool[], same size as input), Valid Poses (Pose[]), $\triangle$ 2.2.x also Angles (Number[], degrees).

Parameters **[D]**: *Reference Frame* (display only: World frame), *Reference Direction Source* (Pose default / Vector3d), *Reference Direction* ($\pm X/\pm Y/\pm Z$ or **Customized**; default **Positive Z**), *Axis to Be Specified* (X/Y/Z default), **Max Angle Difference (0–180, default 90°)**, *Output Empty List if Containing Invalid Poses*, plus visualisation options. Angle $\leq$ threshold → True.

$\triangle$ Parameter table is 2.2.1 text; the 2.1.2 page is marked "under maintenance" — verify against your build **[?]**.

8.5 Validate Existence of Poses in 3D ROI

Ports **[D]**: in Poses; out Validation Results (Bool[]), Validated Poses (Pose[]).

Parameters **[D]**: **3D ROI Name** (with *Open the editor*), **Input Poses Coordinate Type** (Robot frame / Camera frame), *Show Poses Type*. $\triangle$ 2.2.1 text **[?]**.

Two machine-tending uses **[E]**: (1) a **reach/keep-in gate** on the infeed — a pallet corner that drifts outside the usable envelope, or a part that has fallen behind the bin, yields a valid pose the robot must never attempt; (2) an **occupancy check on the machine side** — draw the ROI around the chuck/vise/nest and use Validation Results to answer "is there a finished part to unload?" or "is the nest clear?", then branch with Trigger Control by Flag (§9.2). Watch the frame parameter: an ROI drawn in the robot frame silently rejects everything if poses arrive in the camera frame.

8.6 Sort 3D Poses V2

Ports **[D]**: in Poses; out **Sorted Poses** (Pose[]), **Mapped Indices** (Index[]).

Eleven rules **[D]**: None; **S shape on plane**; **Z shape on plane**; **X / Y / Z value of pose**; Distance from pose to reference pose; Distance from pose to reference pose **on XOY plane**; **Pose confidence values**; Projection of pose in reference direction; Cuboid diagonal lengths. Shape rules take *Row direction* (default +X), *Column direction* (default +Y), **Row Interval** (default 100.000 mm); value/distance rules take *Ascending* (default on); projection takes a *Reference direction* (default +X).

8.7 Filter (2.1.x) / Filter Data by Boolean Value (2.2.x)

Ports **[D]**: in **Filter Criteria** (Bool[]) + **Unnamed** (Abstract[]); out **Unnamed** (Abstract[]). The unnamed ports are literal — the step is type-agnostic, so only the Boolean port is named.

Parameters **[D]**:

Parameter	Notes
Port Number (1–15)	Default 1. Set to 3 or 4 so one Boolean list culls pick points, pick point info, labels and dimensions together
Operation Layer (0–14)	Default 0 = operate on every element
Polarity	2.1.x: Reverse Bool List (unselected = elements whose Boolean is False are filtered out). Δ 2.2.x renamed Keep Only Data with Boolean Value as False , default still unselected
Top Level	2.2.x only, default unselected

Usual predecessors: steps that output Bool[] — *Dichotomize Values by Threshold* (2.1.x) / *Compare Value with Threshold* (2.2.x), *Validate Labels and Output Flags*, the two Validate steps above, *Validate Point Clouds* **[D]**.

Upgrade trap [D]: the polarity parameter was renamed between 2.1.x and 2.2.x while the default stayed "keep the True items". Re-read this checkbox after any upgrade.

8.8 Reorder by Index List

Ports **[D]**: in **Indices** (Index[]) + **Unnamed** (Abstract[]); out **Unnamed**. Worked example: indices 0, 2, 3, 1 applied to 50, 60, 70, 80 → 50, 70, 80, 60.

△ 2.2.1 parameters **[D]**: *Port Number (1–15), Operation Layer (0 = outermost), Top Level.*
2.1.2 documents no parameters.

8.9 Trim Input List

Ports **[D]**: in Unnamed (Abstract[]); out Unnamed, same type. Parameters: *Port Number (1–15), Operation Layer, Output Size (default 1)* — the number of leading items kept.

8.10 Trigger Control by Flag

Ports **[D]**: in **Boolean List** (Bool[]) — *the list should contain only one Boolean value; no data output* — it emits a control-flow signal. Parameter △ 2.2.1: **Trigger Type** = TriggerWhenTrue / TriggerWhenFalse **[D]**.

Use for **per-cycle** logic (nest occupied → unload branch; zero parts → bin-empty branch), and Filter for **per-part** logic **[E]**. Feed it a one-element list, e.g. from a threshold comparison on a count — not a per-part Boolean list.

8.11 Flip Poses' Axes

Ports **[D]**: Poses in, Poses out. Parameters: **Axis to Flip** (X/Y/Z default), **Direction Type** (Negative default / Positive — the target direction of the flipped axis), **Reference Axis to Rotate Around** (X default / Y / Z, **must differ from the axis to flip**), *Pose Type to Visualize*.

Documented behaviour example: with *Axis to Flip* = Z and *Direction Type* = Positive, a pose whose Z axis is < 90° from the positive world axis is untouched, while one > 90° is flipped **180° around the reference axis [D]**.

8.12 Transform Poses

Ports **[D]**: in **Original Poses**, optional **Reference Poses** (required for AllWithFirst, FirstWithAll, UseCorrespondenceInput); out **Transformed Poses**. Transformation types as listed in §7.8. For truss/gantry robots use *Transform Poses for Truss* instead **[D]**.

8.13 Path Planning (Mech-Vision step)

Requires Mech-Viz installed and licensed — the feature is Mech-Viz-related **[D]**.

Documented usage: projects using **Standard Interface or Adapter** communication where only the robot motion *near the vision point* needs planning; after building a scene and inputting vision points, the step outputs a **collision-free** path after point-cloud collision detection **[D]**. Predecessor: pose processing. Successor: the Output step with Port Type = **Predefined (robot path)** (2.1.x) / Output Type = **Robot path for picking** (2.2.x) **[D]**.

Ports — 2.1.2 **[D]**: in Vision Points, Collision Point Cloud, Pose Labels, Object Dimensions, plus Scene Object Names/Dimensions/Poses when *Update Scene Object* is enabled; out **Planned Path, Filter Results** (Bool[]: True = meets requirement). △ 2.2.1 renames/expands **[D]**: in Pick Points (Pose[]), Pick Point Info (JsonValue), Collision Point Cloud (PointCloud[]), Pick Point Labels (String[]), Target Object Dimensions (Size3D[]), Pick Point Offset (Pose[]), scene-object ports; out Planned Path (JsonValue),

Filtered Results (Bool[]). There is **no documented output named "Error Results"** — if your build shows that label, it is the per-vision-point pass/fail Boolean list [?].

Parameters [D]:

Parameter	Machine-tending guidance
Workflow Configuration	Opens the path planning tool; select the configured workflow
Update Scene Object	Enable when the bin/pallet pose is measured each cycle, so collision checking uses the real pose
Select Scenario	2.1.2: Matching (Machine Tending/Positioning/Assembly) (default) — your case, named explicitly. 2.2.1: 3D model matching (Machine Tending/Positioning/Assembly)
Method to Convert Data	<i>Generate picking strategy based on object center point</i> (default; symmetric objects — the step converts centre point to pick point) or <i>based on pick points</i> (multiple pick points needing filtering; you must feed pick point + pick point info)
Point Cloud Type	CloudXYZRGB (default) / CloudXYZ / CloudXYZNormal
Point Cloud in Camera Frame	Selected by default → the cloud is converted to the robot frame before being sent to the tool
Remove Point Cloud of Irregular Shape + Search Radius (default 3 mm)	Enable with a configured collision model when you need collision detection on the held object from pick to place — e.g. a long shaft on its way out through the machine door. Invalid for objects generated from common 3D shapes
Other Inputs / Other Ports	Adds Pick Point Labels, Target Object Dimensions, Pick Point Offsets. If enabled they must receive data , or the Output step fails

The tool plans "*a collision-free motion path near the target object*"; its usage and some parameters **differ from Mech-Viz**, and Mech-Mind explicitly warns to consult each manual separately [D]. You choose the robot model from the Robot Model Library, then work through: configure project resources (tools, target objects, scene objects) → configure workflow → configure collision detection → simulate and debug [D].

Its **default 6-step workflow** is a ready-made machine-tending approach pattern [D]: Set Robot Tool → Above-Bin Fixed Waypoint 1 → Above-Workobject Approach Waypoint (relative move) → **Vision Point** → Above-Workobject Departure Waypoint (relative move) → Above-Bin Fixed Waypoint 2. Add further *Fixed-Point Move* / *Relative Move* steps as needed.

Collision detection [D]: by default collisions between any two of **robot, robot tool, and scene objects** are detected. Enabling *Enable collision detection with point cloud* adds tool-vs-cloud checking, with: **point cloud form** — *Point cloud cube* (fills the cloud surface with cubes) or *Point cloud column* (fills the space beneath the cloud with columns along -Z, extent set by *Column base position*); **tool-point cloud collision volume upper limit**; and

point cloud collision record — *Record* highlights colliding regions in plan history but **slows execution**, *Do not record* is faster but untraceable.

Practice [E]: use *Point cloud column* when parts sit on an opaque pallet the camera cannot see through — otherwise the planner treats the space under the visible surface as free. Keep collision recording **off** in production, on only while chasing a specific collision report.

8.14 Output

Sends the vision result of the current project to Mech-Viz or the communication component. **Only one Output step per project**. It has **no output ports**. Hard rule: *"All default ports must be connected to valid input data; otherwise, an error will occur when running the Output Step"* [D]. Full contract in §11.

8.15 Show Point Clouds and Poses

Ports [D]: in Point Clouds Shown in White, optional Point Clouds Shown in Color, optional Poses; **no data output**. Parameters: *Show Normals*, *Display Interval of Normals* (default 20), *Z Value Visualization Settings* with *Upper/Lower Bound*.

Visualisation convention: **X red, Y green, Z blue**, with an overlay such as Pose List 1: size: 8 [D]. Turning on Z-value visualisation with bounds around the top layer makes multi-layer tray unloading legible — you can see the current layer separate from the one below [E].

8.16 Reading step status, warnings and timing

Signal	Meaning
Step frame colour	Regular / Selected / Warning (yellow frame) / Error [D]
Time: <n>ms in the lower-left of a step block	That step's execution time [D]
Log panel tabs	Vision (project debug info and errors), Console (control info, including the final TCP and status codes output by the vision system), Operation (user actions) [D]
Log levels	D debug, i information, W warning, E error; toggle with the icons at lower-left (blue = shown) [D]
Log retention	Default 7 days , configurable at <i>Settings</i> → <i>Options</i> → <i>Common</i> → <i>Log settings</i> , max 99 days [D]

The yellow "input is empty / no output / auto-fill output" warnings. These exact strings are **not documented** [?]. The reference project shows several of them, which is normal for a project inspected while idle. Practical reading [E]: they are *warnings*, not errors — the step ran, found one of its input lists empty (the "nth input" names the port, 1-based), produced nothing meaningful, and emitted a padded/empty output so downstream steps still receive a well-formed list instead of failing. **They almost always mean an upstream step returned zero items** — recognition found nothing, or a validate+filter chain culled everything. Look upstream, not at the warning step.

Documented mechanisms that behave similarly and are worth knowing [D]:

Execution flag	Behaviour
Continue Given No Output	The project continues even if a step produces no output
Trigger Control Flow Given No Output / Given Output	Whether the control flow fires when the step has no / has output
Notify Procedure Finished When No Output	Whether to notify of empty project output and end the run
Reuse Input	Aligns list lengths between input ports by duplicating the shorter one
Visualize Outputs	Interlocked with the eye icon on the step block

A genuine hard failure looks different: an unconnected **necessary** input port raises a "Procedure Execution Error" pop-up, and the fix is to connect the port [D].

Run vs Continuous Run [D]: *Run* (Ctrl+R) executes the project once. *Continuous Run* repeats it per *Max number of continuous executions* (**-1 = indefinitely**) and *Interval after project execution*, set via the gear beside the button. In production the project is triggered by command 101, not by either button. Use Continuous Run for soak testing: does recognition hold as the tray empties, is cycle time stable, does the camera drop frames over 500 cycles [E].

Debug Output [D]: the panel sits in the upper right. Enable the toolbar toggle and click the **eye icon** on a step (closed → open) to visualise it. Click a step's **Single Step Execution** button to run that step alone; **double-click a data-flow line** to display that stream's contents — the fastest way to inspect an intermediate port. Pop out individual windows to watch several steps at once; the chosen 3D viewpoint is retained on the next run. Documented debugging method: when the result is wrong, "*locate the Step(s) of error by running the Steps one by one*", then tune that step.

9. Fixture-side and cell-state logic

This is the part of machine tending that has no equivalent in bin picking, and the part most often bolted on late.

9.1 What state the cell has

State	How it is known	Owner
Machine cycle complete	M-code / DO from CNC	PLC/robot
Door open / closed	Interlock switch	PLC/robot
Chuck / vise open / clamped	Pressure or proximity switch	PLC/robot
Nest occupied by a	Sensor if fitted; vision ROI check if not	Robot, informed by

State	How it is known	Owner
finished part	(§9.2)	vision
Blank available in the infeed	Vision result count	Vision → robot
Correct part number loaded	Label from recognition, or recipe ID	Vision → robot
Part seated correctly	Seat-check air switch (best), or post-load vision check	PLC/robot

Rule [E]: prefer a hard sensor for anything safety- or damage-critical (seat check, chuck clamped). Use vision for what sensors cannot see — *which* part, *where* it is, and whether the nest is clear when no sensor exists.

9.2 Nest occupancy check with vision

Pattern [E], built entirely from documented steps:

```

Capture (from a pose that sees the fixture)
├─ 3D Target Object Recognition (the finished-part target
object)
├─ Validate Existence of Poses in 3D ROI ROI = the nest volume
│   └─ Validation Results → Trigger Control by Flag
(TriggerWhenTrue)
│   └─ UNLOAD branch: Output the part
pose
└─ (no poses in ROI) → Trigger Control by Flag
(TriggerWhenFalse)
    └─ LOAD branch: nest is clear

```

Notes:

- Trigger Control by Flag needs a **single-element** Boolean list [D] — reduce the per-part Bool[] to a count comparison first.
- With an eye-to-hand camera that cannot see inside the machine, run this check on the *staging position* instead, or fit a sensor. Do not fake it.
- The robot can also learn "nothing to unload" from the protocol: status **1002** (*No vision result*) or **1003** (*No point cloud in ROI*) from command 102 [D]. That is often simpler than a branch (§14.3).

9.3 Load / unload sequencing with one gripper

1. Vision: is the nest occupied? → yes
2. Unload: pick finished part from nest, place in outfeed
3. Vision: find next blank (101 → 102/105)
4. Load: place blank into nest, clamp, verify seat
5. Start machine cycle

With a **double gripper** (archetype A3), steps 2–4 collapse into one trip and the ordering constraint moves into the motion plan — which is the point at which Mech-Viz earns its licence (§10.1).

9.4 Error proofing before the door closes

Cheap checks that prevent expensive crashes **[E]**:

Check	Method
Wrong part number picked	Compare the recognition label against the active recipe; the robot receives the label as element 7 of each vision point [D]
Part missing from the gripper	Gripper part-present sensor (not vision)
Part not seated	Seat-check air pressure switch
Chips in the nest	Air blast + a <code>Validate Existence of Poses in 3D ROI</code> check, or a point-cloud count threshold
Short / oversized part	Enable <i>Point clouds of matched target objects</i> and compare dimensions [D]
Machine still holding a part	§9.2

9.5 Bin-empty and end-of-tray

Three documented ways to tell the robot "nothing left" **[D]**:

1. **Status codes** from 102: **1002** *No vision result*, **1003** *No point cloud in ROI*.
2. **Vision-Move-specific: 1044 / 2039** *No vision point for the Vision Move Step*.
3. **The Notify step + command 601** — a positive-integer message pushed from the project (§12.7).

Prefer 1 [E]: it needs no extra project structure and it is unambiguous to the robot program. Use Notify when you need to distinguish *several* application states ("tray empty" vs "layer complete, index the tray") in one cycle.

10. Motion planning: Path Planning step vs Mech-Viz

10.1 The decision

Use the **Mech-Vision Path Planning step** when

The robot program owns the whole cycle

The vision-dependent motion is a short approach–pick–depart near one bin

You want one fewer piece of software to

Use a full **Mech-Viz project** when

You need branching by part number or fixture state (*Branch by Message* + command 203)

Placement into a chuck/fixture with orientation constraints and symmetry-aware grasp selection

Counters / DO sequencing owned by the vision

Use the **Mech-Vision Path Planning step** when

launch and keep alive on the IPC

Standard Interface or Adapter communication **[D]**

—

Use a full **Mech-Viz project** when

side, or gripper DO lists returned to the robot (206)

Multi-station or dual-gripper cycles

Master-Control — the Path Planning step cannot do master control

Supporting evidence: the step's documented scope is planning "*near the vision point*" / "*near the workobject*" **[D]**; a Mech-Mind community answer (semi-official, **[?]**) states the Path Planning step is "*limited to simple applications*" and non-master-control only, while Mech-Viz enables "*more complex projects (with multiple branches, oriented placing, interaction with adapter etc.)*".

Default for machine tending [E]: start with the **Path Planning step** (archetypes A1, A2, A6). Move to Mech-Viz when you hit A3/A4/A5 complexity. The reference *Drill_Tap* project uses the Path Planning step, which is the right call for a single-station load with a fixed cycle owned by the robot.

10.2 Mech-Viz project structure

A **project** holds **project resources** — robots, tools, workobjects, scene objects — plus the **workflow** (the visual robot-motion program). Reference frames: **World** (fixed at the simulation centre), **Robot Base**, **Robot TCP**, **Scene Object [D]**. A **solution** is the top-level container bundling Mech-Vision project(s), one Mech-Viz project, robot configuration and calibration files **[D]**.

Steps are function modules connected exit-port → entry-port. Categories: *General Parameters of Steps, Basic Move, DI DO, Logical Topology, Pallet, Robot Utilities, Service, Utilities, Trajectory, Vision, Others [D]*.

Canonical pick-and-place workflow order [D]:

Home (move) → Waypoint (move) → visual_look_1 → relative_move_1 (approach, e.g. Z -200 mm)
→ visual_move_1 → Set_DO_Pick (set_do) → wait_1 → relative_move_2 (depart)
→ relative_move_3 (approach place) → move_1 (place) → Set_DO_Place → wait_2
→ relative_move_4 (depart) → Return_Home

Relative Move takes its reference from "**Next**" (upcoming target) or "**Current**" (previous target) **[D]**. Internal names are visual_look / visual_move; the UI labels are *Vision Look / Vision Move [D]*.

For machine tending, extend this skeleton with a branch after visual_look for load vs unload, and put the machine handshake **outside** Mech-Viz wherever possible (§2.3) **[E]**.

Parameter-level references for the *Checker*, *Counter* and *Message* steps were not reachable for this handbook [?]; Branch by Message is confirmed indirectly through command 203 [D].

10.3 Tool, TCP and workobject configuration

Tool [D]: add the tool model (usually **STL**) under *Collision Models*, rotating axes if the import orientation is wrong, then create an entry under *End Effectors* with a name and that model. **TCP** defaults to the flange centre — *"increase the value of Z until the TCP is at the tip of the tool"* — but in real deployments *"please input the calibrated TCP values"*, or use **Update TCP from Robot** to pull the controller's active TCP.

Workobject parameters [D]: **Rotation Symmetry** (how the object coincides with itself under rotation — this drives which pick orientations are legal), **Picking Relaxation** (whether exact positioning is required at grasp), **Optimal Picking Solution Strategy** (how far the tool may rotate). Hard constraint: **the workobject's Z-axis must oppose the TCP's Z-axis during grasping**; symmetric objects *"can be grasped at multiple orientations"*.

Machine-tending note [E]: for turned studs, shafts and drill blanks — bodies of revolution — setting rotational symmetry correctly is what unlocks reachable grasps on a chuck-loading approach. Get it wrong and the planner rejects perfectly good picks as unreachable because it insists on one arbitrary clock position.

Scene objects to model [D]: safety fence, picking bin, tray, camera, camera stand. For machine tending add the machine body, the open door envelope, the chuck/vise, and any tailstock or tool turret that can be inside the swept volume [E].

10.4 Collision detection

Pairs checked **by default and not configurable [D]:** *robot links ↔ robot tool*, and *scene objects ↔ robot tool*.

Tool model format matters [D]:

Format	Geometry	Detection	Thresholds
STL	Mesh (linked triangles)	Surface collisions only	<i>Collision points threshold</i> (default 0), <i>Collision area threshold</i> (default 0)
OBJ	Assembly of convex polyhedra (solid)	<i>"Precise and efficient";</i> detects contact with the tool interior	<i>Collision volume threshold</i> (default 5 mm³)

Caution [D]: *"OBJ models that contain concave polyhedra cannot be used for collision detection anymore."* For a two-jaw chuck-loading gripper, decompose it into convex pieces or accept STL surface-only detection.

Both formats expose a pose preference: *minimum X/Y-axis rotation vs minimum collision within the X/Y rotation range [D]*. Threshold tuning is an explicit trade: higher thresholds

raise pick success but risk disturbing neighbouring parts; lower thresholds guarantee clean paths at a throughput cost [D].

Point cloud collision detection [D]: the cloud is converted to an **octree** of cubes centred on points; the key parameter is "*Resolution of octree converted from point cloud*" (cube edge length; the docs illustrate 10 mm vs 2 mm). Shorter edge = more accurate, slower.

Prerequisite: the Mech-Vision project must include the "Send Point Cloud to External Service" step. Default values and the separate volume/point thresholds for the cloud are not documented [?].

Machine-tending practice [E]:

- Model the gripper **with the part in it** for the loaded moves, or enable *Remove Point Cloud of Irregular Shape* in the Path Planning step so the held part is checked (§8.13).
- Model the machine door aperture generously. The most common real collision in machine tending is the *part*, not the gripper, catching the door frame or the chuck jaws on the way in.
- Simulate at production speed and with a **full** bin and a **nearly empty** one; collision-free at 30 % speed with three parts left proves nothing.

11. The Output step as an interface contract

Treat Output as an interface specification signed by the vision engineer and the robot programmer [E].

11.1 Choose the type by who plans motion

2.1.x Port Type [D]:

Value	Use
Predefined (vision result) — default	Mech-Vision does vision only; result goes to Mech-Viz or an external service. Suits Master-Control, Standard Interface or Adapter
Predefined (robot path)	Mech-Vision does vision and path planning. Only the Path Planning step can feed it, and receiving these results via Mech-Viz is not supported
Custom	Customise the data sent; prefer predefined keys where possible

△ **2.2.x** renames it Output Type with four values: *Vision results for picking* (default), *Vision results for trajectory*, *Robot path for picking*, *Custom* [D].

11.2 Custom port keys (2.1.x)

All array format [D]:

Key	Meaning	Required
-----	---------	----------

Key	Meaning	Required
poses	Object poses	Yes
labels	Object labels, same length as poses	No
sizes	3D dimensions, same length as poses or length 0	No
offsets	Pick-point offset relative to object centre point	No
objectIndexes	Object index	No
scene_object_names / _sizes / _poses	Scene objects to update	No
workobject_data	Target object picking strategy for Mech-Viz; connect to the <i>Picking Strategy</i> output of Generate Picking Strategy	Yes when using Mech-Viz

Interaction rules **[D]**:

- workobject_data alone → only Mech-Viz can parse it; Standard Interface commands **100, 102, 105, 110** get nothing.
- Other ports alone → Mech-Viz cannot parse and errors with *"No pick points have been obtained"*.
- Both → Mech-Viz parses only workobject_data, while the other data still reaches the Standard Interface.

11.3 Other parameters that matter in production

[D] unless noted:

Parameter	Production setting
Send Point Cloud to External Service	Default on; documented as <i>"usually used for debugging or checking the running result"</i> . Turn it off after commissioning to save bandwidth every cycle — unless Mech-Viz point-cloud collision detection needs it (§10.4), in which case it must stay on
Update Scene Object	On when the bin/pallet pose is re-measured, so the same pose informs collision checking and the robot
Point Cloud in Camera Frame	Default on
Point Cloud Type	CloudXYZRGB default
Remove Point Cloud of Irregular Shape + Search Radius (3 mm)	As §8.13
Other Inputs	Adds Pick Point Labels, Target Object Dimensions, Pick Point Offsets
Auto-Correct	Default on; only shown after drift correction is deployed in the

Parameter	Production setting
Accuracy Drift in Vision System	Error Analysis Tool. Keep it on for ± 1 mm class cells

11.4 What the robot actually receives

Critical conversions performed on the way out [D]. Note the actor: items 1 and 2 are performed by the **vision system when it converts the Output step's poses port data into robot TCP for Standard Interface commands 100/102** — they are *not* done by the Output step itself, and poses handed to Mech-Viz remain quaternions.

1. Object poses are converted **from quaternions to Euler angles**.
2. The pose is **rotated 180° about X** to orient its Z-axis downward.
3. *"If the first input port of the Output Step is **Object Center Points**, the Output Step will convert the object center points into the corresponding pick points" — so the poses the robot receives are **pick points**, not centre points.*
4. **Label must be an integer-formatted string**; if no label is available it defaults to 0. A non-integer label raises status **1017**.
5. **Tool ID** defaults to 0; *"in most cases, vision points output by Mech-Vision do not have information of the object tool ID"*. For waypoints from 105/205 the tool ID comes from the path planning tool / Mech-Viz.

Write these five facts into the interface document you hand the robot programmer [E]. Items 2 and 3 in particular explain most "the pose looks 180° out" and "it picks 40 mm above the part" reports.

12. Robot ↔ vision communication (Standard Interface)

12.1 Ground rules

[D] unless noted:

- The **robot / PLC / host is the client**; the vision system is the **server**. The client always initiates; the vision system only replies and **never pushes unsolicited data** (the one exception is the Notify mechanism, §12.7).
- **Formats**: ASCII (comma-delimited, \r terminator, e.g. 103,1,2\r) or HEX (**fixed 64 bytes**, big- or little-endian; short data zero-padded, extra fields ignored).
- **Units**: joint positions in **degrees**; flange/tool pose [x, y, z, a, b, c] with x/y/z in **mm** and a/b/c as **Euler angles in degrees** in the selected brand's convention.
- **Batching**: default **max 20 poses per reply**, configurable in *Robot Communication Configuration* → *Next* → *Advanced Settings*, **hard limit 30**. Default reply **timeout 10 s**.
- **Robot-side buffer**: the robot can hold up to **100** vision points or waypoints from the vision system.

- **Protocol format is brand-pinned:** FANUC V7–V9 = HEX **big-endian**; FANUC V10 = HEX little-endian; ABB (RobotWare 6) = HEX little-endian; KUKA = HEX little-endian; **YASKAWA = ASCII**.
- The interface service must be **running** — after *Apply*, verify the Robot Communication Configuration toggle on the toolbar is flipped and blue, or nothing responds.

12.2 Configuration checklist

In Mech-Vision → *Robot and Communication* → *Robot Communication Configuration* [D]:

1. **Select robot** → *Listed robot* → robot model → Next.
2. **Interface service type** = **Standard Interface**.
3. **Protocol** = **TCP Server**.
4. **Protocol format** per §12.1.
5. **Port number** — free, and see §3.4 for the FANUC V10 ≤ 32767 constraint.
6. *Robot integration* → **Open program folder** — the source of the MM_* files to load onto the controller.
7. Optionally **Auto enable interface service when opening the solution** (recommended for production).
8. **Apply**, then confirm the toolbar toggle.

For TCP Server the **IP address field** is **0.0.0.0** and the robot program needs the **IPC's IP** plus this port [D].

Advanced Settings holds **maximum number of poses to obtain each time** (default 20, limit 30), **Timeout for getting Mech-Vision data** (default 10 s), and **Property Configuration** (the property_config file for commands 207/208) [D].

12.3 Command set — Mech-Vision

All [D], 2.2.1 names. Legacy 1.7.x names in Appendix B.

#	Name	Sent	Reply	Success
100	Trigger Mech-Vision Project and Get Results	100, project ID, recipe ID, returned data format, joint positions, flange pose	Depends on format 1–4	1100
101	Trigger Mech-Vision Project	101, project ID, expected number of vision points or waypoints, robot pose type, robot pose	101, status	1102
102	Get Vision Results	102, project ID	102, status, transmit status,	1100

#	Name	Sent	Reply	Success
			count, reserved, vision point 1 (TCP, label, tool ID), ...	
103	Switch Mech-Vision Parameter Recipe	103, project ID, recipe ID (1-99)	103, status	1107
104	Switch Mech-Vision Solution	104, solution ID	104, status	1104 [?]
105	Get Planned Path from Mech-Vision	105, project ID, waypoint pose type (1 = JPs, 2 = tool pose)	105, status, transmit status, count, position of "Vision Move" in path, waypoint 1 ...	1103
106	Get Gripper DO List from Mech-Vision	106, project ID, number of vacuum grripper sections	106, status, DO 1 ... DO 64	1106
110	Get Custom Data from Mech-Vision	110, project ID	110, status, transmit status, N, pose, label, element 1 ... N	1100
111	Get Vision Move Data from Mech-Vision	111, project ID, waypoint pose type	111, status, transmit status, waypoint type, pose, motion type, tool ID, Vision Move data	1103
501	Input Object Dimensions	501, project ID, length, width, height (mm)	501, status	1108
503	Input Poses to Project	503, project ID, Step name, pose	503, status	1110
504/505	Set / Get Numeric Global Variable	504, ID, value / 505, ID	status (+ value)	1111 [?]
506/507	Set / Get String Global Variable	as above	as above	1111 [?]
601	Get Message from Notify Step	601	601, message	<i>(no status code)</i>
701	Calibration	701, calibration status, flange pose, joint positions	701, status, calibration status, next flange pose, next joint positions	7101

#	Name	Sent	Reply	Success
901	Get System Status	901	901, status, Mech-Vision status, Mech-Viz status, camera status	9100

Command 101 robot pose type [D]:

Type	Pose sent	Scenario
0	0,0,0,0,0,0 — Path Planning start point = the tool's Home point	Eye-to-hand, no pre-capture needed
1	Current JPs and flange pose	Eye-in-hand; recommended for most scenarios except gantry
2	Current flange pose	Recommended for gantry robots
3	User-defined joint positions, used as the Path Planning start point	Eye-to-hand where images are captured beforehand

Command 901 decode [D]: Mech-Vision 0 = not opened / 1 = opened; Mech-Viz likewise; camera 0 = at least one camera disconnected / 1 = all connected. Poll it at cell start-up before the first cycle [E].

12.4 Command set — Mech-Viz

All [D]:

#	Name	Sent	Success
200	Trigger Mech-Viz Project and Get Planned Path	200, Branch-by-Msg Step ID, exit port, waypoint pose type, joint positions, flange pose	2100
201	Trigger Mech-Viz Project	201, robot pose type, robot pose	2103
202	Stop Mech-Viz Project	202	2104
203	Set Exit Port for Branch by Msg	203, Step ID, exit port	2105
204	Set Current Index	204, Step ID, Current Index	2106
205	Get Planned Path from Mech-Viz	205, waypoint pose type	2100
206	Get Gripper DO List from Mech-Viz	206, number of vacuum gripper sections	2102
207 / 208	Read / Set Mech-Viz Step Parameter	207, Config ID / 208, Config ID	2109 / 2108
210	Get Vision Move Data or Custom Data from Mech-Viz	210, returned data format (1–4)	2100

Command 201 robot pose type [D]: 0 = [0,...,0], the simulated robot moves from its set home (eye-to-hand); 1 = current JPs + flange pose (recommended eye-in-hand); 2 = JPs customised by the robot, i.e. a taught image-capture point — *"robot pose type should be set to 2 for projects in eye to hand mode"*, and it allows planning ahead, shortening cycle time.

Two off-by-one traps, both verbatim from the docs [D]:

- 203 / 200 exit port: *"When the parameter value is set to 'N', the Mech-Viz project exits from the port with an ID of 'N-1'."* So 203, 2, 1 selects port **0** of Step 2.
- 204 Current Index: *"When this parameter value is set to 'N', the Current Index of the corresponding Step is 'N-1'."*

207/208 Config IDs come from a property_config file [D]:

```
read, <Config ID>, <Step ID>, <parameter key name>
write, <Config ID>, <Step ID>, <parameter key name>, <parameter value>
```

Examples: read, 5, 3, xCount, write, 6, 3, xOffset, 0.000000. Several write lines may share a Config ID (setting multiple parameters in one command); read IDs must be unique. Key names come from Mech-Viz Tools → Key Query Tool (Developer mode). **The interface service must be restarted after editing property_config [D].**

12.5 Calling sequence — get this wrong and nothing works

[D], verbatim constraints:

Mech-Vision chain: 103 and/or 501 (before) → **101** → one of 102 / 105 / 110. *"Command 102, Command 105, and Command 110 cannot be used at the same time."*

Mech-Viz chain: 207/208 (before) → **201** → 204 then 203 (**204 before 203**) → 205 or 210 → after the project stops, 206. *"Command 205 and Command 210 cannot be used at the same time."* For index-type steps the prescribed order is **201 → 204 → 203** *"to ensure that Mech-Viz has enough time to set the Current Index value."*

Other ordering constraints [D]: 104 before 100/101; 106 after 100/105/111; 601 *"immediately AFTER Command 101 or Command 201"*; 503 before 101.

PLC handshake (PROFINET / EtherNet/IP / Snap7 / MC / Modbus), verbatim [D]: write command number + parameters to registers → PLC sets Command_Trigger = 1 → vision reads → vision sets Trigger_Acknowledge = 1 → PLC sets Command_Trigger = 0 → vision sets Trigger_Acknowledge = 0.

12.6 Reading a vision result

102, status, transmit status, count, reserved, vision point 1 ... [D]:

- **transmit status** — 0 = not all vision points obtained; 1 = all obtained. *"If the value is 0, repeat sending the command until the value becomes 1. If not all vision points are obtained when Command 101 is called, the remaining vision points will be cleared."*
- **count** — the number in *this* reply (≤ 20 by default, 30 max).

- **reserved** — not in use, value 0.
- **vision point** — exactly **8 elements**: 1–6 = TCP, 7 = label, 8 = tool ID.

Documented 22-point exchange [D]:

```
→ 101, 1, 0, 1, -0, -20.6323, -107.8121, -0, -92.8181, 0.0016
← 101, 1102
→ 102, 1      ← 102, 1100, 0, 20, 0, 95.7806, 644.5677, 401.1013, 31.1206, ...
→ 102, 1      ← 102, 1100, 1, 2, 0, 315.2017, 592.1261, 399.6052, 126.1960, ...
```

Custom data is *not* available through 102 — use 110, which returns **one** vision point per call; call it repeatedly for several points. N = the sum of the column counts of all custom ports, arranged in **alphabetical order of port names**, and the Output step's Port Type must be **Custom** with the poses port present [D].

For 105/205, note the documented semantic shift of *position of "Vision Move" in planned path* across multiple replies: in the first response it is the position within the **entire** path; in later responses it is the position among the **remaining** waypoints [D]. Also set 101's expected count to **0** before using 105, or you must call 105 once per waypoint [D].

motion type: **1 = joint (MOVE)**, **2 = linear (MOVEL)**. tool ID = -1 means no tool used at that waypoint [D].

Terminology correction [D]: there is no parameter called sendPose. The robot-side parameter is **SendPos_Type** (YASKAWA sendpos_type) and it selects *which robot pose is sent to the vision system* (0–3). Batching is governed by Pos_Num_Need, the Advanced Settings pose cap, and the transmit-status loop.

12.7 Notify — the only vision→robot push

There is **no "empty tray" command [D]**. To let the project tell the robot "condition X occurred", use the **Notify** step with command 601:

- **Mech-Vision side [D]**: connect Notify to the right of another step (e.g. Output); in the Output step's parameters select *Trigger Control Flow Given Output*; in the Notify step set **Service Name = Standard Interface Notify** (a required literal) and **Message = a positive integer** (e.g. 1001).
- **Mech-Viz side [D]**: place Notify in the workflow, select **Standard Interface** as the receiver, set Message to a positive integer.
- Reply is 601, <message> — **no status code [D]**.
- **Timing hazard, verbatim [D]**: "*When the Notify Step is executed in the Mech-Vision or Mech-Viz project, the message remains in the buffer of the vision system for only **three seconds**.*" Hence the requirement to call 601 immediately after 101 or 201.
- Range **6001–6199** is documented as "*status codes that can be customized in Mech-Vision*" — the sanctioned block for application states such as "tray empty" [D]; how to author them is not documented [?].

Practice [E]: the 3-second buffer makes Notify fragile in a cell where the robot may be mid-move. Prefer status codes (§9.5) for anything the robot must not miss, and reserve Notify for advisory messages such as "layer complete, index the tray".

13. Robot-side programs

13.1 The universal shape

All four brands follow the same pattern **[D]**:

- A **background/driver module** (socket + protocol) plus thin wrapper routines named `MM_*` called from the user's motion program.
- Results land in **registers / variables**, never as return values.
- **Two-stage retrieval is mandatory:** a *Get* command fetches data into robot memory, then a *Store* command copies one indexed item into a position register or variable. FANUC states it explicitly: *"The returned vision result is saved to the robot memory and cannot be directly obtained. To access the vision result, you must store the vision result in a subsequent step."*
- Programs live on the IPC at Communication Component/Robot_Interface/<BRAND>/, reachable via *Robot Communication Configuration* → *Robot integration* → *Open program folder*; examples in `.../<BRAND>/sample/` **[D]**.

13.2 Generic machine-tending cycle (brand-neutral pseudocode)

```
# ---- once, at controller start / program init ----
MM_INIT_SOCKET(ipc_ip, port, timeout)
MM_GET_STATUS(status)                # cmd 901
IF status != 9100 -> alarm "vision system not ready"
IF vision_status != 1 OR camera_status != 1 -> alarm

# ---- per part ----
MOVE J   home
WAIT    DI[machine_cycle_done] AND DI[door_open]

# --- unload the finished part (taught position or vision) ---
GOSUB   unload_finished_part

# --- select the recipe for the active part number (optional) ---
MM_SWITCH_MODEL(project=1, recipe_id=active_recipe, status)  # cmd 103
IF status != 1107 -> alarm

# --- trigger vision from the capture pose ---
MOVE L   capture_pose                # eye-to-hand: not required; see
pose type 0/3
MM_START_VIS(project=1, pos_num_need=0, sendpos_type=2, pr=..., status)  #
cmd 101
```

```

IF status != 1102 -> GOTO vision_error

# --- fetch either poses or a planned path, never both ---
IF using_path_planning_step:
    MM_GET_VISPATH(project=1, jps_pos=1, pos_num, vis_pos_num, status)    #
cmd 105
    IF status != 1103 -> GOTO vision_error
    FOR i = 1 TO pos_num
        MM_GET_JPS(i, PR[60], R[label], R[toolid])    # store one waypoint
        IF i == vis_pos_index THEN close_gripper    ENDIF
        MOVE (motion_type_i) PR[60]
    ENDFOR
ELSE:
    MM_GET_VIS(project=1, pos_num, status)    # cmd 102
    IF status != 1100 -> GOTO vision_error
    MM_GET_POS(1, PR[60], R[label], R[toolid])    # store vision point
1
    IF R[label] != expected_label_for_recipe -> GOTO wrong_part
    MOVE J approach_via_offset(PR[60], z=+100)
    MOVE L PR[60]    slow
    close_gripper
    MOVE L approach_via_offset(PR[60], z=+100)
ENDIF

# --- load the machine ---
MOVE J door_entry
MOVE L nest_approach
MOVE L nest_seat    slow
open_gripper / release
WAIT    DI[seat_check_ok]    # hard sensor, not vision
SET    DO[chuck_close]
WAIT    DI[chuck_clamped]
MOVE L nest_retreat
MOVE J clear_of_door
SET    DO[cycle_start]
JUMP    top_of_loop

vision_error:
    # decode the status code and branch – see §14
wrong_part:
    # place the part back / to a reject chute, then retry

```

Notes [E]: keep MM_INIT_SOCKET out of the cycle loop — it belongs in initialisation. Keep the IF status != ... check after **every** MM_* call; the example programs do this, and it is the difference between a cell that stops safely and a cell that moves to a stale pose.

13.3 FANUC

Prerequisites [D]: controller system software V7.5, 7.7, 8.x, 9.x or 10.1. **R651 or R632 (KAREL) must be installed. R648 (User Socket Msg) must be installed.** Ethernet to motherboard **CD38A (Port 1)** or **CD38B (Port 2)**.

Loading [D]: copy *the contents* of the FANUC folder (not the folder) to the root of a ≤ 32 GB FAT32 flash drive; on the TP: MENU → FILE → F5 UTIL → Set Device → USB Disk (UD1:) or USB on TP (UT1:), select **INSTALL**, ENTER, F4 YES. Success message: "**Programs Loaded**". Test with **MM_COMTEST** (args: robot port 1–8, IPC IP, IPC port, timeout in **minutes**). Changing the robot port number for the first time requires a controller restart. The individual installed file names are not published [?].

Routines (exact signatures) **[D]:**

Feature	Routine
Init communication	MM_INIT_SKT(C_Tag, Ip_Addr, Svr_Port, Time_Out) — C_Tag is a string '1'–'8'; Time_Out in minutes ; V10 Svr_Port 1–32767
Run Mech-Vision project	MM_START_VIS(Job, Pos_Num_Need, SendPos_Type, Pr_Num, MM_Status)
Get vision result	MM_GET_VIS(Job, Reg_Pos_Num, MM_Status)
Store result/path (TCP)	MM_GET_POS(Serial, Pr_Num, Reg_Label, Reg_ToolId) — Serial is 1-based
Store path (joint positions)	MM_GET_JPS(Serial, Pr_Num, Reg_Label, Reg_ToolId)
Switch parameter recipe	MM_SET_MOD(Job, Model_Num, MM_Status)
Get planned path (Mech-Vision)	MM_GET_VISP(Job, Jps_Pos, Reg_Pos_Num, Reg_VPos_Num, MM_Status)
Get / store custom data	MM_GET_DY_DT(Job, Reg_Pos_Num, MM_Status) / MM_GET_DYPOS(Serial, Pr_Num, Reg_Label, Reg_UserData)
Get / set gripper DO list	MM_GET_DL(Resource, Block_Num, MM_Status) / MM_SET_DL(Loop_Index)
Run / stop Mech-Viz	MM_START_VIZ(SendPos_Type, Pr_Num, MM_Status) / MM_Stop_Viz(MM_Status)
Set branch / index	MM_SET_BCH(Branch_Num, Export_Num, MM_Status) / MM_SET_IDX(Skill_Num, Index_Num, MM_Status)
Get planned path (Mech-Viz)	MM_GET_VIZ(Jps_Pos, Reg_Pos_Num, Reg_VPos_Num, MM_Status)
Vision Move / plan data	MM_GET_PLNDT(...) / MM_GET_PLJOP(Serial, Jps_Pos, Pr_Num, Reg_MoveType, Reg_ToolNum, Reg_Speed, Reg_UserData, Reg_PlanRes)
Read / set Viz step	MM_GET_PROP(Read_id, MM_Status, Reg_Viz_Prop) /

Feature	Routine
parameter	MM_SET_PROP(Write_id, MM_Status)
Input object	MM_SET_BS(Job, Length, Width, Height, MM_Status) /
dimensions / poses	MM_SET_VISP(Job, Step_Name, Pr_Num, MM_Status)
Notify / calibration / status	MM_GET_NTIFY(MM_NotifyMsg) / MM_CALIB(...) / MM_GET_STAT(MM_Status)

Canonical TP flow, from the official MM_S1_Vis_Basic [D]:

```

UFRAME_NUM=0 ; UTOOL_NUM=1 ;
J P[1] 100% FINE ; ! home
CALL MM_INIT_SKT('8','127.0.0.1',30000,5) ; ! once only
L P[2] 1000mm/sec FINE ; ! image-capture pose
CALL MM_START_VIS(1,0,2,10,53) ;
IF (R[53]<>1102),JMP LBL[99] ;
CALL MM_GET_VIS(1,51,53) ;
IF R[53]<>1100,JMP LBL[99] ;
CALL MM_GET_POS(1,60,70,80) ; ! PR[60]=TCP, R[70]=label,
R[80]=toolId
J P[3] 50% CNT100 ;
L PR[60] 1000mm/sec FINE Tool_Offset,PR[1] ; ! approach
L PR[60] 300mm/sec FINE ; ! pick
...
LBL[99:vision error] ;

```

FANUC ships 21+ example programs, MM_S1_Vis_Basic ... MM_S22_Vis_As_Uframe [D]. The ones worth reading for machine tending: **MM_S6_Viz_ErrorHandle** (error handling), **MM_S9_Viz_RunInAdvance** (pipelined capture to cut cycle time), **MM_S13_Vis_MoveInAdvance**, MM_S15_Viz_GetDoList, MM_S19/S20_PlanAllVision, MM_S8/S10_Viz_Subtask, MM_S21_1Robot_3IPC_Sequentially.

13.4 ABB (RAPID)

Prerequisites [D]: IRC4 or IRC5; **RobotWare 6.02–6.15**; control module **616-1 PC Interface**. Ethernet to controller **X6 (WAN)**.

Files loaded to task T_ROB1 [D]: MM_Module.mod (program module), MM_Auto_Calib.mod (calibration), MM_Com_Test.mod (comms test). **RobotWare 7: change the extension from .mod to .modx**. Auto-load via Communication Component\tool\Robot Program Loader (back up first, then *Load the Standard Interface program* → *Load with one-click*), or manually via the TP / RobotStudio.

ABB is the only brand with **explicit open/close socket routines [D]**.

Feature	Routine
Init / open / close	MM_Init_Socket IP_Address, Server_Port, Time_Out; (Time_Out in seconds) · MM_Open_Socket; · MM_Close_Socket;
Run project / get	MM_Start_Vis Job, Pos_Num_Need, SendPos_Type, MM_J,

Feature	Routine
result	MM_Status; ·MM_Get_VisData Job, Pose_Num, MM_Status;
Store pose / JPs	MM_Get_Pose Serial, MM_P, MM_Label, MM_ToolId; ·MM_Get_Jps Serial, MM_J, MM_Label, MM_ToolId;
Recipe / planned path	MM_Switch_Model Job, Model_Number, MM_Status; ·MM_Get_VisPath Job, Jps_Pos, Pos_Num, VisPos_Num, MM_Status;
One-shot trigger + fetch	MM_Lite_Vis Job, Model_Number, Recv_Data_Type, MM_Status, \MM_J, \Pos_Num, \VisPos_Num; — max 20 poses
Custom data	MM_Get_DyData job, Pos_Num, MM_Status; ·MM_Get_DyPose Serial, MM_P, MM_Label;
DO list	MM_Get_DoList Resource, BlockNum, MM_Status; ·MM_Set_DoList Loop_Index, Serial, Go16;
Mech-Viz	MM_Start_Viz, MM_Stop_Viz, MM_Set_Branch, MM_Set_Index, MM_Get_VizData, MM_Lite_Viz, MM_get_plandata, MM_Get_PlanPose
Properties / dimensions	MM_Get_Property, MM_Set_Property, MM_Set_BoxSize
Notify / calib / status	MM_Get_Notify Msg, MM_Status; ·MM_Calib Move_Type, Pos_Jps, Wait_time \Ext; ·MM_Get_Status MM_Status;

Recv_Data_Type for the Lite routines: 1 = vision points without custom data (then MM_Get_Pose); 2 = with custom data (then MM_Get_DyData); 3 = waypoints as joint positions (then MM_Get_Jps); 4 = waypoints as TCP (then MM_Get_Pose) [D].

Canonical RAPID core [D]:

```

MoveAbsJ home\NoEOffs, v3000, fine, gripper1;
MM_Init_Socket "127.0.0.1", 50000, 300;          ! once only
MoveL camera_capture, v1000, fine, gripper1;
MM_Open_Socket status;
IF status = 3099 THEN TPWrite "MM: Communication Error"; STOP; ENDIF
MM_Start_Vis 1, 0, 2, snap_jps, status;
IF status <> 1102 THEN TPWrite "MM: Status Error"; STOP; ENDIF
MM_Get_VisData 1, pose_num, status;
IF status <> 1100 THEN Stop; ENDIF                ! 1003 no cloud in ROI / 1002
no result
MM_Close_Socket;
MM_Get_Pose 1, pickpoint, label, toolid;
MoveJ pick_waypoint, v1000, z50, gripper1;
MoveL RelTool(pickpoint, 0, 0, -100), v1000, fine, gripper1;
MoveL pickpoint, v300, fine, gripper1;

```

Note the pattern: **open socket** → **commands** → **close socket** → **then store poses**. The docs warn that leaving the socket open for a long time can produce an "abnormally closed" error [D].

13.5 KUKA (KRL)

Prerequisites [D]: 6-axis; **KR C4 or KR C5**; **KSS 8.2 / 8.3 / 8.5 / 8.6 / 8.7**; add-on **Ethernet KRL**, version pinned to KSS:

KSS	Ethernet KRL
8.2 / 8.3	2.2.8
8.5	3.0.3
8.6	3.1.2.29
8.7	3.2.2.16

Ethernet port: **X66** (KR C4 Compact), **KLI** (other KR C4), **XF5** (KR C5). Expert mode required (default password kuka) **[D]**.

Files and destinations [D]: mm_module.src + .dat → **KRC:\R1\mm** (create mm if absent); MM_COMTEST.src + .dat → same; XML_Kuka_MMIND.xml → **C:\KRC\ROBOTER\Config\User\Common\EthernetKRL**; sample/ for examples.

```
<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL><IP>192.168.1.121</IP><PORT>50000</PORT></EXTERNAL>
    <INTERNAL><ALIVE Set_Flag="873"/></INTERNAL>
  </CONFIGURATION>
  <RECEIVE><RAW><ELEMENT Tag="MMRecv" Type="BYTE" Set_Flag="871" Size="660"
/></RAW></RECEIVE>
  <SEND><RAW><ELEMENT Tag="MMSend" Type="BYTE" Size="660"/></RAW></SEND>
</ETHERNETKRL>
```

<IP> = the IPC, <PORT> = the configured host port; flag **873** = connected (ALIVE), flag **871** = data received; payload buffer **660 bytes** each way **[D]**.

Routines **[D]**: MM_Init_Socket(XML_Name, Alive_Flag, Recv_Flag, Time_Out) — e.g. MM_Init_Socket("XML_Kuka_MMIND", 873, 871, 60), Time_Out in **seconds**, XML_Name case-sensitive without extension. Then MM_Start_Vis, MM_Get_VisData, MM_Get_Pose, MM_Get_Jps, MM_Switch_Model, MM_Get_Vispath, MM_Get_Dy_Data, MM_Get_DyPose, MM_Get_DoList, MM_Set_DoList, MM_Start_Viz, MM_Stop_Viz, MM_Set_Branch, MM_Set_Index, MM_Get_VizData, MM_Get_PlanData, MM_Get_PlanPose, MM_Get_Property, MM_Set_Property, MM_Set_BoxSize, MM_Get_Notify, MM_Calib, MM_Get_Status.

KUKA-only and very useful for machine tending [D]: MM_Get_Wobj(Serial, MM_Frame_num, MM_Label, MM_ToolId) stores the vision pose directly into a **base coordinate variable** — e.g. MM_Get_Wobj(1, 10, label, toolid) writes base variable 10. That lets you express the whole insertion motion as a base-relative sub-program, which is exactly how you want a chuck-loading move written **[E]**.

13.6 YASKAWA (INFORM + MotoPlus)

Prerequisites [D]: 6-axis. **DX200** → **DN3.16.00A-00**, **YRC1000** → **YAS2.94.00-00**, **YRC1000micro** → **YBS2.31.00-00**. **ETHERNET option must be USED** and MotoPlus **APPLI**. **AUTOSTART AT POWER ON must be ENABLE**. Ethernet: YRC1000 → **LAN2 (CN106)** on the CPU board (LAN1 is pendant-only; LAN3 if LAN2 is occupied); DX200 → **CN104**.

Files [D]: mm_module_ycr1000.out / mm_module_dx200.out (MotoPlus background application), JBI/ (the foreground MM_* jobs), sample/. Load the .out in maintenance mode via MotoPlus APL. → DEVICE USB: Pendant → LOAD(USER APPLICATION); verify in FILE LIST. Load jobs via EX. MEMORY FOLDER → JBI → EX. MEMORY LOAD → JOB → EDIT → SELECT ALL. **Only one .out may be loaded** on most controllers — delete the existing one first or you get FATAL:MM:SOCKET_OPEN_ERROR. Test job **MM_COMTEST** (edit line 0001 for IP/port; default port 50000).

Arguments are passed as a **single semicolon-delimited string** via ARGF [D]:

Feature	Call
Init / open / close	MM_INIT_SOCKET("IP;Port;Time_Out") (Time_Out in minutes) · MM_OPEN_SOCKET · MM_CLOSE_SOCKET
Run project / get result	MM_START_VIS("job;pos_num_need;sendpos_type;prNum;regStatus") · MM_GET_VISDATA("Job;Pose_Num;MM_Status")
Store pose / JPs	MM_GET_POSE("Index;Robtarget;Label;Tool_Id") (P variable type must be robot) · MM_GET_JPS("Index;Jointtarget;Label;Tool_Id") (joint/pulse)
Recipe / path	MM_SET_MODEL(...) · MM_GET_VISPAT("job;GetPos_Type;Pos_Num;VisPos_Num;regStatus")
Custom data / DO list	MM_GET_DYDATA, MM_GET_DYPOSE, MM_GET_DOLIST, MM_SET_DOLIST
Mech-Viz	MM_START_VIZ, MM_Stop_Viz, MM_SET_BRANCH, MM_SET_INDEX, MM_GET_VIZDATA, MM_GET_PLANDATA, MM_GET_PLANPOSE
Properties / dimensions / notify / calib / status	MM_GET_PROPERTY, MM_SET_PROPERTY, MM_SET_BOXSIZE, MM_GET_NOTIFY, MM_CALIB, MM_GET_STATUS

Canonical INFORM core [D]:

```
NOP
CLEAR I050 20
SUB P070 P070 / SUB P071 P071
SETE P070 (3) 100000 ' 100 mm Z offset
(micron units)
MOVJ C00000 VJ=50.00
CALL JOB:MM_INIT_SOCKET ARGF"192.168.170.22;50000;1" ' once only
```

```

MOVJ C00001 VJ=50.00 PL=0           ' image-capture pose
CALL JOB:MM_OPEN_SOCKET
CALL JOB:MM_START_VIS ARGF"1;0;2;30;80"
IFTHENEXP I080<>1102 / PAUSE / ENDIF
CALL JOB:MM_GET_VISDATA ARGF"1;51;52"
IFTHENEXP I052<>1100 / PAUSE / ENDIF   ' 1003 no cloud in ROI,
1002 no result
CALL JOB:MM_CLOSE_SOCKET
CALL JOB:MM_GET_POSE ARGF"1;71;61;62"
MOVJ C00002 VJ=50.00
SFTON P070 / MOVL P071 V=166.6 PL=0 / SFTOF   ' approach via shift
MOVL P071 V=50.0 PL=0                       ' pick
END

```

13.7 Cross-brand differences that bite

All **[D]**:

Difference	Detail
Time_Out units	FANUC and YASKAWA = minutes ; ABB and KUKA = seconds
Explicit socket open/close	ABB and YASKAWA yes ; FANUC and KUKA no
Protocol format	FANUC V7–V9 HEX big-endian; FANUC V10 HEX little-endian; ABB/KUKA HEX little-endian; YASKAWA ASCII
Port range	FANUC V10: 1–32767 ; others recommend 50000+
Robot-side error strings	FANUC: MM:Robot_Internal_Error [X], MM:Robot_Socket_Closed, MM:Robot_Argument_Error [X], MM:Robot_CMD_Error. YASKAWA: FATAL:MM:INTERNAL_ERROR:..., SOCKET_OPEN_ERROR, SOCKET_CONNECT_ERROR, SOCKET_SEND_ERROR, SOCKET_SELECT_ERROR, SOCKET_RECV_ERROR, SOCKET_RECV_TIMEOUT, SOCKET_CLOSED, ARGUMENT_MISSING, ARGUMENT_INVALID

MM:Robot_CMD_Error means the command code was not found, or the code received by the background program does not match what the vision system sent — i.e. **a calling-sequence violation (§12.5) [D]**.

14. Error handling and recovery

14.1 Design the failure modes first

A machine-tending cell that stops on every anomaly has poor OEE; one that retries blindly breaks tools. Classify every failure into one of four responses **[E]**:

Class	Response	Examples
-------	----------	----------

Class	Response	Examples
Retry — transient, no state change	Re-trigger vision, up to N times, then escalate	Capture timeout, 1019 execution timeout, 1047 capture-completion timeout
Skip — this part is unusable, others may not be	Discard this pose, take the next; if none left, treat as class <i>Wait</i>	1030 unreachable waypoint, 1036 collision detected, 1035 invalid pick point
Wait — nothing to do, but nothing is broken	Signal the operator/PLC, hold, poll	1002 no vision result, 1003 no point cloud in ROI, 1044/2039 no vision point
Stop — integrity is in doubt	Controlled stop, alarm, require human intervention	1023 failed to connect to camera, 3099 socket failure, 1015 project execution error, any unexpected code

Write this table into the robot program as an explicit dispatch, not as a chain of IF status <> 1100 THEN PAUSE. FANUC's own MM_S6_Viz_ErrorHandle example is the pattern to follow [D].

14.2 Status code dispatch (machine-tending subset)

Code	Meaning [D]	Class [E]	Robot action [E]
1100	Get vision result successfully	—	Proceed
1102	Project triggered successfully	—	Proceed to fetch
1103	Obtained planned path successfully	—	Execute waypoints
1106/1107/1108	DO list / recipe / dimensions OK	—	Proceed
1002	No vision result	Wait	"Infeed empty" to PLC; hold; poll
1003	No point cloud in ROI	Wait	Same; also check for a displaced bin
1007	Project is running	Retry	Wait 200 ms, retry once; if persistent → Stop (double-trigger bug)
1011	Project ID does not exist	Stop	Configuration error
1012/1013/1014	Recipe ID missing / not configured / switch failed	Stop	Changeover configuration error (§16)
1015	Project execution error	Stop	Look in the Vision log Console tab for the CV-Exxxx detail
1016	Data does not match the Output step's Port Type	Stop	You called the wrong command for the Output

Code	Meaning [D]	Class [E]	Robot action [E] type (§11.1)
1017	Failed to map label string to numbers	Stop	Labels must be integer-formatted [D]
1019	Execution timeout	Retry	Retry once, then Stop
1020	Project not started	Stop	Calling-sequence violation — 102 before 101
1023	Failed to connect to camera	Stop	Camera / network / power fault
1026	Invalid pose type	Stop	Bad robot pose type argument
1027	Runtime error at Path Planning step	Stop	Planning configuration fault
1030	Robot cannot reach the waypoint	Skip	Next pose; count consecutive failures
1033	Motion singularity error	Skip	Next pose
1035	Invalid pick point	Skip	Next pose
1036	Robot collision detected	Skip	Next pose; if all fail, escalate — likely a mis-modelled scene
1044	No vision point for the Vision Move step	Wait	As 1002
1046	Invalid tool	Stop	Tool config mismatch
1047	Timeout waiting for capture completion	Retry	Retry; if repeated, check the camera link speed
2001–2047	Mech-Viz equivalents (Appendix A)	as above	Mirror the Mech-Vision mapping
3099	Failed to open socket (<i>used in every official example program, but absent from the official code table</i>) [?]	Stop	Network / interface service not running
3005	Communication Component: Mech-Vision timed out	Retry→Stop	
3007	Data acknowledge from client timed out	Stop	Fieldbus handshake fault (§12.5)
4002	Robot: Euler angle type not supported	Stop	Wrong brand/convention selected
9100	System status obtained	—	Check the three sub-status fields

Full tables in Appendix A.

14.3 Distinguishing "empty" from "broken"

The single most valuable thing the robot program can do [E]:

```
CASE status OF
  1100:          -> normal
  1002, 1003, 1044: -> DO[infeed_empty] = ON ; message "Load parts" ; wait
; retry
  1030,1033,1035,1036: -> pose_retry = pose_retry + 1
                        IF pose_retry < 3 THEN retry vision ELSE escalate
  ELSE:          -> DO[vision_fault] = ON ; alarm with the code number
ENDCASE
```

Show the numeric status code on the pendant or HMI. A maintenance technician who can read "1003" off the screen finds the displaced bin in a minute; "Vision Error" costs an hour.

14.4 Retry policy that does not break tools

[E]:

- **Never retry a motion into the machine.** Retry only the *vision* portion.
- Cap consecutive vision retries (3 is typical) and cap consecutive skip events (e.g. 5 poses) before escalating; a cell that skips 40 poses is telling you the scene model is wrong.
- After any Skip-class failure, **re-capture** rather than reusing the old result — the scene may have shifted when the previous attempt disturbed the pile.
- On any Stop-class failure, retract to a **known safe position outside the machine envelope** before alarming, if the robot is not already there and it is safe to move.
- Log every failure with its code, timestamp and cycle counter. Vision faults cluster; the pattern is the diagnosis.

14.5 Where to look when it goes wrong

Symptom	First look
Status code returned	Appendix A, then the Vision log Console tab (<i>"the final TCP and status codes output by the vision system"</i>) [D]
1015 with no detail	Vision log Console tab for CV-Exxxx; the interface only reports the generic 1015 [D]. (CV-E0000 specifically is not documented — do not guess a meaning [?])
Robot alarms MM:Robot_CMD_Error	Calling sequence (§12.5) [D]
Yellow step warnings	Look upstream for an empty list (§8.16)
Poses look 180° out	The documented 180°-about-X rotation and quaternion→Euler conversion applied when Output-step poses become robot TCP for commands 100/102 (§11.4) [D]; then Flip Poses' Axes configuration (§8.11)

Symptom	First look
Picks are consistently offset	Calibration residual (§6.5), then TCP, then pick point definition
Works cold, drifts warm	Thermal drift — deploy drift auto-correction [D]
Intermittent, worse when the tray is nearly empty	Depth range or 3D ROI clipping the bottom layer (§5.4)

15. Cycle time engineering

15.1 The published decomposition

Mech-Mind's cycle-time guide decomposes the cycle into **image capturing → vision processing → path planning & robot motion**, and gives two hard numbers [D]:

- The Capture Images from Camera step *"usually takes about **40 %** of the time"* — that is 40 % of the **vision project's runtime**, not of the whole cell cycle. (The overall-approach page publishes no percentage split for the full cycle.)
- **Wait time for vision results should be < 50 ms** *"to ensure that the system can respond quickly."*
- Normal camera data transmission: **700–800 Mbps** — verify the link is truly gigabit.

15.2 Levers, in the order that pays

All [D] unless noted:

Capture side (the largest single share of vision-project time)

1. Minimise exposure time; use **Reflective** fringe coding rather than stacking 2–3 exposures where the camera supports it.
2. Update camera firmware.
3. **Trim 3D ROI and depth range.**
4. Ensure a gigabit link (IPC LED orange/green, §3.3).
5. Last resort: change camera.

Vision processing

6. Do not wire the 2D colour port if it is unused.
7. Use 3D ROI extraction to drop excess cloud; **downsample** where density is not critical.
8. Coarse-to-fine 3D matching.
9. **Disable debug output in production** — and turn off *Send Point Cloud to External Service* unless collision detection needs it (§11.3).
10. Non-deep-learning steps scale with **CPU multi-core**, not GPU — spec the IPC accordingly.

Planning and motion

11. Enable "**Capture-and-Move**" — *"robot to move immediately after camera exposure"* — so processing overlaps motion. This is the biggest structural win in an eye-to-hand cell.
12. **Reuse one vision result for multiple picks** where the scene does not change between them (a kitted tray is the ideal case).
13. **Simplify gripper and scene models** to cut collision computation.
14. Optimise intermediate waypoints and **reduce 6th-axis rotation**.
15. **Increase blend radius** on non-critical moves.

Protocol level [E]

16. Use robot pose type = 2 with 201, or type 3 with 101, so planning starts from a *taught* capture pose and can run **before** the robot arrives — the docs explicitly note this "allows planning ahead and shortens cycle time" [D].
17. Set 101's expected count to 0 and fetch once, rather than looping 105 per waypoint [D].
18. Trim the result list (§7.9): fewer vision points means less planning work.

15.3 A realistic budget

For a NANO ULTRA single-pick machine-tending cell [E], with the documented capture time of 0.5–0.9 s [D]:

Segment	Typical	Notes
Capture	0.5–0.9 s	Reflective mode single pass; 2× exposure roughly doubles it
Vision processing	0.3–1.5 s	The reference project runs 37 steps in 1.50 s total
Path planning	0.1–0.5 s	Grows with vision point count and model complexity
Robot motion (bin → machine → bin)	4–15 s	Dominated by distance and machine door geometry
Machine handshake waits	0.5–3 s	Chuck, door, seat check

Vision is rarely the bottleneck once the capture is tuned — but it is always the thing blamed, because it is the newest component. **Measure and publish the split at commissioning** so later arguments are about data [E].

16. Changeover and multiple part numbers

16.1 Parameter recipes — the documented mechanism

Purpose: *"switch between parameter recipes in one project to make it applicable to various scenarios"* [D].

Create [D]: *Project Assistant* tab → Parameter Recipe panel → settings icon → **Add Recipe**, name it. Then select a **Step** in the left panel and choose **that step's parameters** on the right to enrol them in the recipe.

Switch [D]:

- **Manually** — the dropdown in the Parameter Recipe panel; *"will take effect when you run the project."*
- **Automatically** — with Standard Interface or Adapter, *"you should run a recipe-switching command on the robot side, and Mech-Vision will switch the parameter recipe automatically according to the command it received"*. Use **command 103** (or pass the recipe ID as an argument to 201). **You must specify the recipe ID, not the name**; IDs are the leftmost column of the Parameter Recipe Editor table, positive integers 1–99.

Which parameter types are eligible, and whether there is a cap on recipe count, are not documented [?].

16.2 Choosing a changeover strategy

Strategy	When	Mechanism
One project, parameter recipes	Same part family, differing dimensions/tolerances; same fixture	103 before 101 [D]
One project, multiple target objects	Genuinely different geometries in the same tray	Recognise all, use labels to route; the <i>Select Target Object</i> dropdown selects one, so several projects or several recognition steps may be needed [E]
Multiple projects in one solution	Different cameras, ROIs or capture parameters per part	101 with a different project ID
Multiple solutions	Different cells or radically different applications	104 Switch Mech-Vision Solution [D]
Operator-driven	Manual changeover, no PLC integration	Production Interface target object management (§17.1) [D]

16.3 Changeover checklist

For each new part number, record and verify [E]:

- ☐ Target object: point cloud model, symmetry, **pick points**, collision model.
- ☐ Camera configuration parameter group (finish may differ).
- ☐ 3D ROI and depth range (part height differs → depth range differs).
- ☐ Angle gate threshold (a short stubby part tolerates more tilt than a long shaft).
- ☐ Sort rule and row interval (tray pitch differs).
- ☐ Trim Input List output size.

- ☐ Path planning scene objects (different fixture or nest).
- ☐ Gripper recipe / DO mapping on the robot side, keyed to the label.
- ☐ Recipe ID assigned and documented — the robot program sends the **ID**.
- ☐ Fixture/nest teach positions on the robot.
- ☐ First-off validation: 10 consecutive successful load cycles.

Keep a changeover matrix in the project folder: part number → recipe ID → target object → gripper program → nest teach position. This document is what makes the cell operable by someone other than its author **[E]**.

17. Deployment and production

17.1 Production Interface

The operator-facing runtime shell, without the full development IDE. Documented capabilities **[D]**:

- Real-time production status, production results, project execution status.
- **Production logs and alert records.**
- **Vision system accuracy drift monitoring.**
- **Target object management** — switch between target object types during production, add new target objects, show target object detail. This is the operator-level part-number changeover surface for machine tending.
- **User authentication** (login/logout, account switching, password reset) and solution access control.
- **Backup file management**, in-interface troubleshooting, operation-guide access.

Set it up with the **Production Interface Configurator**; entry is via login **[D]**. Default accounts, roles and passwords are not documented on the reachable pages **[?]** — establish and record your own.

Practice [E]: configure the Production Interface *before* handover and train the operator on exactly three actions: read the status, switch the part number, and export a log for support. Do not leave the development IDE as the production UI; an operator with the full step graph will eventually change a parameter.

17.2 Autoload and start-up

- **Autoload Solution** exists as a documented feature (alongside Create/Save Solution, solution access control, switching rules, and project autoload) **[D]**; the exact semantics — auto-open only, or auto-open plus auto-run — are unconfirmed **[?]**.
- **There is no documented mechanism for launching Mech-Vision / Mech-Viz automatically on Windows boot [?]**. Autoload governs what loads once the application starts, not application launch. If a cell must survive an unattended power

cycle, arrange application launch yourself (a Windows startup shortcut or scheduled task) and **prove it by pulling the power at commissioning [E]**.

- Enable *Auto enable interface service when opening the solution* in Robot Communication Configuration so the TCP server is live without an operator clicking the toggle **[D]**.
- Have the robot poll **901** before the first cycle after any restart and refuse to run unless Mech-Vision, Mech-Viz (if used) and the camera all report ready **[E]**.

17.3 Licensing

- Mech-Mind offers **both** a hardware **dongle** and a **software licensing code [D]**.
- **Plug full license dongles into USB ports before installing software version 1.6.0 or above [D]**.
- Path Planning and Mech-Viz features require **Mech-Viz installed and licensed [D]** — a cell that plans in Mech-Vision still needs the Mech-Viz licence.
- Activation procedure, validity checking, and behaviour on dongle removal are not documented on the reachable pages **[?]**. **Practice [E]**: fit an internal USB port or a locking cover so the dongle cannot be borrowed, and record the licence details in the handover pack.

17.4 Backups

Only the Production Interface's **backup file management** is documented **[D]**; there is no separate backup procedure page **[?]**. Treat this as practice **[E]**:

Back up	When	Where
Whole solution folder (Mech-Vision projects, Mech-Viz project, robot config, Calibration folder)	Before every change; after every accepted change	Versioned, off-machine, with a dated folder name
Camera configuration parameter groups	With the solution	
Target object definitions	With the solution	
Robot program (full controller backup / image)	Before and after every change	Brand-standard backup
property_config (if using 207/208)	With the solution	
Calibration result + error report screenshot	After each calibration	Keeps a record of drift over the cell's life
Known-good reference: step count, execution time, calibration error	At acceptance	The regression baseline

The documented project file structure includes `vision_project.vis`, a Calibration folder, and config files such as `depth_background.png`, `depth_roi.json`, `roiBoundary.json` **[D]** — copy the folder, not selected files.

17.5 Handover pack

[E] — a machine-tending cell should be handed over with:

1. This handbook, with the cell-specific appendix filled in (Appendix E).
2. Network diagram: IPs, ports, subnets, protocol format, NIC assignments.
3. Interface document: which commands the robot sends, in what order, with what arguments, and the status-code dispatch table as implemented.
4. Changeover matrix (§16.3).
5. Calibration record with the error figure and date.
6. Cycle-time split as measured (§15.3).
7. Backup media plus restore instructions.
8. Spares list: camera, dongle, cables, calibration board.
9. Operator one-pager: start, stop, changeover, what the status codes on the HMI mean, when to call for help.
10. Support-ticket template: software versions, camera model/ID/firmware, IPC spec, language settings, log export, and a project that reproduces the issue — the documented list of what Mech-Mind support needs [D].

17.6 Upgrades — read before you touch a running cell

Documented cautions [D]:

- **1.7.0: project migration is mandatory** — projects built with earlier versions must be migrated after upgrading.
- **1.7.2:** legacy deep-learning steps are automatically replaced with *Deep Learning Model Package Inference*.
- **Version lock:** Mech-Center 1.7.0 must be used with Mech-Vision and Mech-Viz 1.7.0 or above.
- **Mech-Viz 1.7.0 renamed robot models** → you must **re-select the robot from the robot model library** after upgrading.
- **Target objects configured in 2.1.2 cannot be used with software earlier than 2.1.2.**
- Step categories, the Filter → Filter Data by Boolean Value rename, the polarity parameter rename, and Port Type → Output Type all changed between 2.1.x and 2.2.x.

Backup requirements, downgrade support, and the Mech-Eye SDK ↔ Suite compatibility matrix are not stated [?].

Practice [E]: never upgrade a producing cell without (a) a full backup, (b) a scheduled window, (c) a re-run of the calibration accuracy check, (d) a re-verification of collision models and robot model selection, and (e) a 50-cycle soak before releasing to production. On a machine-tending cell an invalidated robot model silently changes collision behaviour — the failure mode is a crash, not an error message.

18. Commissioning and acceptance

18.1 Vision acceptance

#	Test	Pass criterion
V1	Camera link speed	1 Gbps or better (IPC LED orange/green) [D]
V2	Point cloud completeness, worst-case part	Complete cloud on graspable surface and matching features
V3	Point cloud stability	≤ 3 mm fluctuation over 10 static captures [D]
V4	Calibration error	< 2 mm at 100 % for machine tending [D]
V5	Euler-angle consistency along the calibration path	< 1° [D]
V6	Recognition rate, full tray	≥ 99 % over 100 captures [E]
V7	Recognition rate, last layer / near-empty	≥ 99 % over 50 captures [E]
V8	False positive rate	0 accepted poses on an empty tray [E]
V9	Empty-tray behaviour	Returns 1002/1003, not a fault [D]
V10	Execution time	Within the takt budget; record the number
V11	Metadata alignment	Label and dimensions correct for each pose across 20 sorted results [E]
V12	Sort order	Matches the documented tray sequence, operator-legible

18.2 Motion acceptance

#	Test	Pass criterion
M1	Simulated cycle, full bin	Collision-free at production speed
M2	Simulated cycle, near-empty bin	Collision-free
M3	Collision model fidelity	Gripper, held part, machine door, chuck, fence all modelled
M4	Reach at the extremes of the bin	No singularity, no axis limit
M5	Held-part collision check	Enabled (§8.13) and demonstrated with a long part
M6	Dry cycle, no part	Full cycle including handshake, no collision
M7	Single-part cycle	Loads, seats, clamps, machine starts

18.3 Integration acceptance

#	Test	Pass criterion
I1	901 at start-up	All three sub-statuses report ready
I2	Calling sequence	No MM:Robot_CMD_Error over 100 cycles
I3	Status dispatch	Each class in §14.1 demonstrated by injecting the

#	Test	Pass criterion
		condition
I4	Empty infeed	Cell holds, signals, resumes when refilled — no alarm
I5	Unreachable pose	Skips to the next pose; escalates after the cap
I6	Camera unplugged mid-cycle	Controlled stop, alarm with code, robot safe outside the machine
I7	IPC power cycle	Cell recovers to a runnable state without a laptop (§17.2)
I8	Wrong part in the tray	Detected by label, rejected, no machine damage
I9	Changeover	Recipe switch by ID, first-off correct, 10 good cycles
I10	Soak	500 cycles (or one shift) with recorded pick success and cycle time

18.4 Acceptance metrics to publish

[E], since Mech-Mind publishes only a ">99 % *picking success rate*" claim for machine tending and **no cycle-time or accuracy figures [D]**:

- Pick success rate over the soak (target $\geq 99\%$).
- Mean and worst-case cycle time, with the vision/motion split.
- Recognition rate and false-positive rate.
- Calibration error at acceptance, with the date.
- Number of operator interventions per shift, by cause.

19. Maintenance and troubleshooting

19.1 Routine schedule

[E], informed by the documented environmental constraints [D]:

Interval	Task
Every shift	Clean the camera window; confirm the camera bracket witness mark; check the infeed for chips; read the Production Interface drift indicator
Weekly	Review the fault log by status code; check pick success trend; verify the IPC has free disk space
Monthly	Calibration accuracy spot check (a taught reference part / calibration sphere); export and archive logs; verify backups restore
Quarterly	Recheck TCP; inspect gripper wear (compliance change moves the effective pick point); review the changeover matrix against reality
Annually	Full recalibration; controller and IPC backup refresh;

Interval	Task
	review software version against Mech-Mind release notes
After any impact, robot repair, robot zeroing, camera swap, or tooling change	Redo hand-eye calibration and re-run §18.1 V4–V5 [E]

19.2 Log locations

[D]:

Component	Path
Mech-Vision	\logs under the Mech-Vision install dir (or <i>Open folder</i> in the Log panel)
Mech-Viz	\logs under the Mech-Viz install dir
Mech-Center (legacy)	\logs under the Mech-Center install dir
Deep learning (.dlkpack / .dlkpackC)	\dl_sdk_log under the Mech-Vision install dir
Deep learning via Mech-Center	\dl_sdk_log under the Mech-Center install dir
Deep learning (legacy .pth / .dlkmp)	\resource\deeplearning_server\logs under Mech-Vision
Crash dumps	.dmp files in the Mech-Vision / Mech-Viz / Mech-Center install dirs
Mech-Eye Viewer software log	Mech-Eye SDK install path (<i>Open log folder</i>)
Camera device log	Mech-Eye Viewer → <i>Device log</i> tab (plain or encrypted)

Camera log detail [D]: levels [i] / [W] / [C] / [F]; filenames encode timestamps (00105_20221117171503_887.log); a trailing .1 marks the **earlier** file; camera storage is limited and old logs are deleted, so **export promptly**. Sync the camera time via Camera Controller on first use.

Mech-Vision log retention defaults to **7 days**, configurable up to **99 days** at *Settings* → *Options* → *Common* → *Log settings* [D]. On a production cell, raise it — a fault that recurs every three weeks is invisible with a 7-day window [E].

19.3 Troubleshooting playbook

Symptom	Likely cause	Check, in order
Recognition rate falls over weeks	Lens film; drift; lighting change	Clean lens → drift indicator → re-run V2/V3 → calibration check
Recognition fails only on the bottom layer	Depth range or 3D ROI clipping	§5.4; re-tune with an empty tray

Symptom	Likely cause	Check, in order
Recognition fails only on the top layer of a full tray	Overexposure at short range; depth range upper limit	§5.2 exposure; depth range
Random pose 180° out	Symmetry / axis normalisation	Flip Poses' Axes config (§8.11); target object symmetry (§10.3)
Robot picks 40 mm high or low	Pick point definition; TCP; calibration	Pick point in the target object editor → TCP → calibration error
Right pose, wrong gripper program	Metadata misalignment	§7.7 — check every Sort/Filter/Trim has matching Reorder/port count
Works in simulation, collides in reality	Scene model incomplete; point cloud collision off; opaque pallet	§10.4; enable point cloud collision; use <i>Point cloud column</i>
Intermittent 1019 / 1047	Camera link, exposure stack, IPC load	Link speed → reduce exposures → check IPC CPU load and Windows Update state
MM:Robot_CMD_Error	Calling-sequence violation	§12.5; check for a stray second 101 or a 102/105 mix
Cell hangs waiting for vision	Interface service off; socket left open; timeout mismatch	Toolbar toggle blue? → socket open/close pattern (ABB/YASKAWA) → Time_Out units per brand (§13.7)
Everything worked yesterday, nothing today, after an upgrade	Robot model renamed; step renames; target object version	§17.6
Nuisance faults after a Windows restart	Windows Update re-enabled; firewall re-enabled; app not launched	§3.3, §17.2

19.4 Getting help

Export the log and open a ticket with: software versions, camera model / ID / firmware, IPC spec, language settings, the exported log, and a project that reproduces the issue **[D]**. For a 1015, include the CV-Exxxx line from the Vision log **Console** tab — the interface itself cannot tell you more **[D]**.

Appendices

Appendix A — Status codes

Ranges **[D]**:

Range	Category
1001–1099	Mech-Vision error codes
1100–1199	Mech-Vision normal status codes
2001–2099	Mech-Viz error codes
2100–2199	Mech-Viz normal status codes
3001–3099	Communication Component error codes
3100–3199	Communication Component normal status codes
4001–4099	Robot error codes
4100–4199	Robot normal status codes
6001–6199	Status codes that can be customised in Mech-Vision
7001–7099	Calibration error codes
7100–7199	Calibration normal status codes

There is no 5xxx range in the Standard Interface code space **[D]**. In 1.6/1.7 the 3xxx block was labelled *Mech-Center*; in 2.x it is *Communication Component* **[D]**.

A.1 Mech-Vision errors (1001–1099) **[D]**

Code	Description
1001	Unregistered project existed in the solution
1002	No vision result
1003	No point cloud in ROI
1005	Invalid command parameter to start Mech-Vision project
1006	Invalid pose data
1007	Project is running
1008	Digital output signal list not provided
1010	Number of poses and number of labels do not match
1011	Project ID does not exist
1012	Parameter recipe ID does not exist
1013	Parameter recipe not configured
1014	Failed to switch parameter recipe
1015	Project execution error
1016	Data obtained by the command does not match the Port Type value of the Output Step

Code	Description
1017	Failed to map the string of label to numbers
1018	Invalid pose number input
1019	Execution timeout
1020	Project not started
1021	Failed to set object dimensions
1022	Invalid object dimension settings
1023	Failed to connect to camera
1024	Number of poses and list size of custom port data do not match
1025	Visual image data in use; running cannot proceed
1026	Invalid pose type
1027	Runtime error at Path Planning Step
1030	Robot cannot reach the waypoint
1033	Motion singularity error
1035	Invalid pick point
1036	Robot collision detected
1037	No available placing position for palletizing
1044	No vision point for the Vision Move Step
1046	Invalid tool
1047	Timeout occurs when waiting for the capture completion
1048	Box mask recognition error
1049	Check box dimensions error
1051	Failed to set pose
1052	Failed to declare/get global variable
1053	Failed to execute the switch solution instruction

A.2 Mech-Vision normal (1100–1199) [D]

Code	Description
1100	Get vision result successfully
1101	Ready to run
1102	Project got triggered successfully
1103	Obtained planned path successfully
1106	Obtained DO signal list successfully
1107	Parameter recipe switched successfully
1108	Object dimensions input successfully
1110	Pose set successfully
1104 /	Solution switched / global variable set-or-read — used on the command pages

Code	Description
1111	but absent from the official status table [?]

A.3 Mech-Viz errors (2001–2099) [D]

Code	Description
2001	Software not registered
2002	Project is running
2004	Robot cannot reach the waypoint
2006	Invalid command parameter to start Mech-Viz
2008	Runtime error
2011	Digital output signal list not provided
2012	Invalid pose type
2013	Invalid pose data
2014	No project set to autoload
2015	Project not opened
2016	Failed to set Step parameter value
2017	Failed to stop execution
2018	Invalid branch exit port number
2019	Failed to set branch
2020	Motion singularity error
2022	Project not executed, or no results after execution
2024	Invalid Branch Step ID
2025	Execution timeout
2026	Invalid Step ID of index-type Steps
2027	Invalid Current Index value
2028	Failed to set index
2030	Invalid pick point
2031	Robot collision detected
2032	No available placing position for palletizing
2036	Visual Recognition Step not called
2037	No vision result received from vision service
2038	No point cloud in ROI
2039	No vision point for the Vision Move Step
2041	Failed to get Step parameter
2042	Failed to get planning result in Vision Move
2043	Failed to get custom data
2044	Vision service not registered

Code	Description
2045	Invalid tool
2047	Check box dimensions error

A.4 Mech-Viz normal (2100–2199) [D]

Code	Description
2100	Execution completed successfully
2102	DO signal list obtained successfully
2103	Started successfully
2104	Stopped successfully
2105	Set branch successfully
2106	Set index successfully
2107	Set move point for External Move Step successfully
2108	Set Step parameter value successfully
2109	Get Step parameter value successfully

A.5 Communication Component, robot, calibration [D]

Code	Description
3001	Invalid command
3002	Invalid data length or format for command parameter
3005	Mech-Vision timed out (gRPC service call)
3006	Unknown error
3007	Data acknowledge signal from client timed out
3008	Configuration ID does not exist
3103	Modbus TCP data cleared successfully
3099	Failed to open socket — used in every official example program but absent from the official table [?]
4002	Robot: Euler angle type not supported
7001	Calibration: parameter error
7002	Calibration: no calibration flange pose provided
7003	Calibration: calibration joint positions not provided
7004	Calibration: robot failed to reach the calibration point
7100	Calibration: robot moved to the calibration point successfully
7101	Calibration: pose received from Mech-Vision successfully

3007 fires when, on PROFINET / EtherNet/IP, the client fails within the timeout (default 10 s) to reset Data_Acknowledge to 0 before new pose data is sent, set it to 1 after, clear NOTIFY via CLEAR_NOTIFY before the next notify message, or reset EXPOSURE_COMPLETE via RESET_EXPOSURE before the next exposure [D].

Legacy 1.6 codes dropped in 2.x but still seen in field notes: 3003 client disconnected, 3004 server disconnected, 3100 client connection normal, 3101 server connection normal, 3102 waiting for client to connect [D].

Appendix B — Command quick reference and legacy names

#	2.x name	1.7.x name
100	Trigger Mech-Vision Project and Get Results	<i>(2.x addition)</i>
101	Trigger Mech-Vision Project	Start Mech-Vision Project
102	Get Vision Results	Get Vision Target(s)
103	Switch Mech-Vision Parameter Recipe	Switch Mech-Vision Recipe
104	Switch Mech-Vision Solution	<i>(2.x addition)</i>
105	Get Planned Path from Mech-Vision	Get Result of Step "Path Planning"
106	Get Gripper DO List from Mech-Vision	<i>(2.x addition)</i>
110	Get Custom Data from Mech-Vision	Get Custom Output Data from Mech-Vision
111	Get Vision Move Data from Mech-Vision	<i>(2.x addition)</i>
200	Trigger Mech-Viz Project and Get Planned Path	<i>(2.x addition)</i>
201	Trigger Mech-Viz Project	Start Mech-Viz Project
202	Stop Mech-Viz Project	(same)
203	Set Exit Port for Branch by Msg	Select Mech-Viz Branch
204	Set Current Index	Set Move Index
205	Get Planned Path from Mech-Viz	Get Planned Path
206	Get Gripper DO List from Mech-Viz	Get DO List
207 / 208	Read / Set Mech-Viz Step Parameter	(same)
210	Get Vision Move Data or Custom Data from Mech-Viz	Get Waypoint with Depalletizing Planning Data
501	Input Object Dimensions	(same)
502	—	Input TCP to Mech-Viz (External Move); absent from the 2.2.1 list , though status 2107 survives [?]

#	2.x name	1.7.x name
503–507	Input Poses / global variables	<i>(2.x additions)</i>
601	Get Message from Notify Step	Notify
701	Calibration	(same)
901	Get System Status (3 status fields)	Get Software Status (1101 = ready)

All **[D]**.

Appendix C — Robot routine cross-reference

Function	FANUC	ABB	KUKA	YASKAWA
Init comms	MM_INIT_SKT	MM_Init_Socket	MM_Init_Socket	MM_INIT_SOCKET
Open / close socket	—	MM_Open_Socket / MM_Close_Socket	—	MM_OPEN_SOCKET / MM_CLOSE_SOCKET
Trigger Mech-Vision	MM_START_VIS	MM_Start_Vis	MM_Start_Vis	MM_START_VIS
Get vision result	MM_GET_VIS	MM_Get_VisData	MM_Get_VisData	MM_GET_VISDATA
Store pose (TCP)	MM_GET_POS	MM_Get_Pose	MM_Get_Pose	MM_GET_POSE
Store joint positions	MM_GET_JPS	MM_Get_Jps	MM_Get_Jps	MM_GET_JPS
Switch recipe	MM_SET_MOD	MM_Switch_Model	MM_Switch_Model	MM_SET_MODEL
Get planned path (Vision)	MM_GET_VISP	MM_Get_VisPath	MM_Get_Vispath	MM_GET_VISPATH
Custom data	MM_GET_DY_DT / MM_GET_DYPOS	MM_Get_DyData / MM_Get_DyPose	MM_Get_Dy_Data / MM_Get_DyPose	MM_GET_DYDATA / MM_GET_DYPOSE
DO list	MM_GET_DL / MM_SET_DL	MM_Get_DoList / MM_Set_DoList	MM_Get_DoList / MM_Set_DoList	MM_GET_DOLIST / MM_SET_DOLIST
Trigger / stop Mech-Viz	MM_START_VIZ / MM_Stop_Viz	MM_Start_Viz / MM_Stop_Viz	MM_Start_Viz / MM_Stop_Viz	MM_START_VIZ / MM_Stop_Viz
Branch / index	MM_SET_BCH / MM_SET_IDX	MM_Set_Branch / MM_Set_Index	MM_Set_Branch / MM_Set_Index	MM_SET_BRANCH / MM_SET_INDEX
Get planned path (Viz)	MM_GET_VIZ	MM_Get_VizData	MM_Get_VizData	MM_GET_VIZDATA
Step parameter	MM_GET_PROP /	MM_Get_Property /	MM_Get_Property /	MM_GET_PROPERTY /

Function	FANUC	ABB	KUKA	YASKAWA
r/w	MM_SET_PROP	MM_Set_Property	MM_Set_Property	MM_SET_PROPERTY
Object dimensions	MM_SET_BS	MM_Set_BoxSize	MM_Set_BoxSize	MM_SET_BOXSIZE
Notify	MM_GET_NTFY	MM_Get_Notify	MM_Get_Notify	MM_GET_NOTIFY
Calibration	MM_CALIB	MM_Calib	MM_Calib	MM_CALIB
System status	MM_GET_STAT	MM_Get_Status	MM_Get_Status	MM_GET_STATUS
One-shot trigger+fetch	—	MM_Lite_Vis / MM_Lite_Viz	—	—
Pose → base frame	—	—	MM_Get_Wobj	—
Comms test program	MM_COMTEST	MM_Com_Test.mod	MM_COMTEST.src	MM_COMTEST

All **[D]**.

Appendix D — Naming and project hygiene conventions

[E] — adopt or replace, but write it down and follow it:

Item	Convention	Example
Solution	<Cell>_<Machine>	MechMaster
Vision project	<Part>_<Operation>	Drill_Tap
Target object	<partnumber>	t1 → prefer PN12345_blank
Camera parameter group	<part>_<finish>	t1_machined_reflective
Calibration group	`_<ETH	EIH>`
Step instance	<Role>_<Detail>	Gate_Tilt_PreFlip, Sort_Layer_Z, Trim_OnePerCycle
3D ROI	<Region>	ROI_Infeed, ROI_Nest
Recipe	<PartNumber> with a fixed ID	PN12345 = ID 3
Robot program	MT_<Cell>_<Function>	MT_LATHE1_MAIN, MT_LATHE1_VISION
Register block	Reserve and document a contiguous block	R[50–59] status, PR[60–69] vision poses

Appendix E — Cell-specific record (fill in per installation)

Item	Value
Cell / machine	
Archetype (§1.2)	

Item	Value
Robot make / model / controller / software version	
Camera model / SN / firmware	
IPC model / OS / software versions (Vision, Viz, Eye SDK)	
Mounting mode (ETH/EIH) and working distance	
Camera IP / IPC IP / robot IP / subnet	
Standard Interface port / protocol format	
Solution name / project name(s) / project IDs	
Recipe table (ID → part number)	
Calibration date / method / error at 100 %	
Tolerance budget (§1.3)	
Measured cycle time and split (§15.3)	
Known-good step count / execution time	
Backup location	
Licence type / dongle serial	
Acceptance date / signed by	

Appendix F — Items to verify on your build

Every claim marked [?] in this handbook, collected:

1. CV-E0000 — no documented meaning; CV-Exxxx is a Mech-Vision internal step error surfaced as status 1015 only.
2. `exit_code` — no such field exists anywhere in the Standard Interface; if your project emits one, it is an Adapter-layer or step-level field.
3. 3099 (failed to open socket) — used in every official example program, absent from the official status table.
4. 1104 and 1111 — used on the command pages, absent from the official status table.
5. No 5xxx status range exists.
6. Command 502 (Input TCP to Mech-Viz) — present in 1.7.4, absent from the 2.2.1 list, although status 2107 survives.
7. UDP Standard Interface command reference — officially "*not yet available*".
8. 3D Target Object Recognition port labels *Pick Point IDs* / *Colored Point Cloud* / *Merged Point Cloud* / *Object Point Clouds* / *Pick Point Labels* — not documented under those names; probable mapping in §7.4.
9. .tob target object file extension — not documented anywhere reachable.
10. "Target Object Confidence" as a parameter name — not documented; see the *Target object confidences* port and 3D Matching's *Result Verification Degree + Confidence Threshold* (default 0.3).
11. Path Planning output labelled "*Error Results*" — docs say *Filter Results* (2.1.2) / *Filtered Results* (2.2.1).

12. The yellow warnings "The nth input is empty. No output. Auto-fill output" / "No output. Auto-fill output" — not documented; nearest documented mechanisms in §8.16.
13. A per-run "step count" line format in the Vision log — not documented; only per-step Time: n ms on the block is.
14. Parameters for *Validate Poses by Included Angles to Reference Direction*, *Validate Existence of Poses in 3D ROI*, *Easy Create Poses*, *Trigger Control by Flag*, *Reorder by Index List* — marked "under maintenance" in 2.1.2; tables here come from 2.2.1.
15. Calibration board selection table by camera model / working distance.
16. No official "recalibrate when the camera is bumped" statement exists; Quick Camera Replacement prerequisites are undocumented.
17. Licence dongle activation, validity check, and removal behaviour.
18. No documented Windows autostart for Mech-Vision / Mech-Viz; *Autoload Solution* semantics unconfirmed.
19. Solution backup procedure — only the Production Interface's backup file management is documented.
20. Mech-Viz *Checker*, *Counter*, *Message* step parameter references.
21. Point-cloud collision default values and the separate volume/point thresholds.
22. Mech-Eye per-model parameter defaults (Gap Filling, Fringe Contrast Threshold, Outlier Removal, Surface Smoothing, Distortion Correction, Anti-Blur).
23. Upgrade backup requirement, downgrade path, and the Mech-Eye SDK ↔ Suite compatibility matrix.
24. Machine-tending cycle times and mm accuracy are **not published** by Mech-Mind; only a ">99 % picking success rate" claim.
25. Production Interface default accounts, roles and passwords.
26. A Solution Library template literally named "machine tending" or "CNC loading" is not confirmed; the confirmed equivalents are **General Workpiece Recognition** and the tutorial's two **Vision-Guided Loading ...** templates.
27. NANO ULTRA mounting mode — documented as eye-in-hand in the model comparison while the machine-tending solution page recommends PRO/LSR; confirm ETH support for your SKU. Also confirm **which variant** you have (350M, 250–500 mm vs 700M, 400–800 mm) — the case label on the reference unit does not say.
28. Parameter recipe eligibility rules and any cap on recipe count.
29. Which 2.1.x build corresponds to the observed 2.1.1 UI in the reference project — this handbook uses 2.1.2 documentation as the closest published reference.

Appendix G — Glossary

Term	Meaning
Vision point	An object recognised by Mech-Vision: pose, label, dimensions, custom data [D]
Waypoint	A point the robot reaches along a planned path: pose, label, motion type [D]

Term	Meaning
Pick point	A grippable position defined in the object reference frame [D]
Target object	The configured object definition: point cloud model, symmetry, centre point, pick points, collision model [D]
Parameter recipe	A named set of step parameters switchable at runtime by ID [D]
Solution	Container for Mech-Vision project(s), one Mech-Viz project, robot config, calibration [D]
Standard Interface	Mech-Mind's fixed command protocol where the external device is the client [D]
Master-Control	Mode where the vision system commands the robot [D]
Adapter	Mode where you define the protocol and formats yourself (Python) [D]
ETH / EIH	Eye-to-hand (fixed camera) / eye-in-hand (camera on the flange) [D]
Communication Component	The service hosting the Standard Interface server; replaced Mech-Center in 2.x [D]
Notify	The step + command 601 mechanism for pushing a message to the robot; 3-second buffer [D]
Mapped Indices	The permutation emitted by a sort step, applied via Reorder by Index List [D]

Appendix H — Sources

Standard Interface and robot integration

- TCP/IP Interface Commands (2.2.1) — <https://docs.mech-mind.net/en/robot-integration/latest/standard-interface-development-manual/tcp-ip-socket.html>
- TCP/IP Interface Commands (1.7.4) — <https://docs.mech-mind.net/en/robot-integration/1.7.4/standard-interface-development-manual/tcp-ip-socket.html>
- Status Codes and Troubleshooting — <https://docs.mech-mind.net/en/robot-integration/latest/standard-interface-development-manual/status-codes-error-troubleshooting.html>
- Calling Sequence of Standard Interface Commands — <https://docs.mech-mind.net/en/robot-integration/latest/standard-interface-development-manual/control-timing.html>
- Standard Interface Development Manual (protocol matrix) — <https://docs.mech-mind.net/en/robot-integration/latest/standard-interface-development-manual/standard-interface-development-manual.html>
- Communication Modes — <https://docs.mech-mind.net/en/robot-integration/latest/communication-basics/communication-modes.html>
- Robot Communication Configuration — <https://docs.mech-mind.net/en/suite-tutorial/latest/vision-system-communication-configuration.html>

- FAQ 13, IP address and port — <https://docs.mech-mind.net/en/robot-integration/latest/faq/faq-13.html>
- MC / Standard Interface commands (2.1.2) — <https://docs.mech-mind.net/en/robot-integration/2.1.2/standard-interface-development-manual/mc-commands.html>
- FANUC: commands · setup · examples · MM_S1_Vis_Basic · error messages — <https://docs.mech-mind.net/en/robot-integration/latest/standard-interface-robot/fanuc-interface-commands.html>
- ABB: commands · setup · MM_S1_Vis_Basic · error messages — <https://docs.mech-mind.net/en/robot-integration/latest/standard-interface-robot/abb-interface-commands.html>
- KUKA: commands · setup · error messages — <https://docs.mech-mind.net/en/robot-integration/latest/standard-interface-robot/kuka-interface-commands.html>
- YASKAWA: commands · setup · MM_S1_Vis_Basic · error messages — <https://docs.mech-mind.net/en/robot-integration/latest/standard-interface-robot/yaskawa-interface-commands.html>

Mech-Vision steps and tools

- Step Reference Guide (2.1.2) — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/steps.html>
- Step Reference Guide (latest) — <https://docs.mech-mind.net/en/suite-software-manual/latest/vision-steps/steps.html>
- Capture Images from Camera — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/capture-images-from-camera.html>
- 3D Target Object Recognition · tool · target object selection · general settings — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/3d-target-object-recognition.html>
- 3D Matching basic tuning (confidence) — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/3d-matching-basic-parameters.html>
- Target Object Editor · Point Cloud Models and Pick Points — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-tools/target-object-editor.html>
- Easy Create Poses — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/easy-create-poses.html>
- Validate Poses by Included Angles to Reference Direction — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/validate-poses-by-included-angle-to-reference-direction.html>
- Validate Existence of Poses in 3D ROI — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/validate-existence-of-poses-in-3d-roi.html>
- Sort 3D Poses V2 — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/sort-3d-poses-v2.html>
- Filter / Filter Data by Boolean Value — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/filter.html>

- Reorder by Index List — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/reorder-by-index-list.html>
- Trim Input List — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/trim-input-list.html>
- Trigger Control by Flag — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/trigger-control-by-flag.html>
- Flip Poses' Axes — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/flip-poses-axes.html>
- Transform Poses — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/transform-poses.html>
- Path Planning step · Path Planning Tool · Configure collision detection — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/path-planning.html>
- Output (procedure out) — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/procedure-out.html>
- Show Point Clouds and Poses — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-steps/show-point-clouds-and-poses.html>
- Step Common Parameters · Statuses of Steps · Input/Output of Steps — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-operation-guide/understand-step-common-parameters.html>
- Run Project and Debug · Run Steps and View Outputs · Run Project Continuously · Project Toolbar · Log — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-operation-guide/run-project-and-debug.html>
- Project file structure — <https://docs.mech-mind.net/en/suite-software-manual/2.1.2/vision-operation-guide/project-file-structure.html>
- Parameter Recipe Configuration — <https://docs.mech-mind.net/en/suite-software-manual/latest/vision-tools/parameter-recipe-configuration.html>
- Guide to Production Interface — <https://docs.mech-mind.net/en/suite-software-manual/latest/vision-production-interface/production-interface.html>
- Auto-Correct Accuracy Drift · Quick Camera Replacement — <https://docs.mech-mind.net/en/suite-software-manual/latest/vision-tools/accuracy-error-analysis-tool-drift-correction.html>

Calibration

- Hand-Eye Calibration Guide — <https://docs.mech-mind.net/en/suite-software-manual/latest/vision-calibration/calibration.html>
- Hand-Eye Calibration tutorial — <https://docs.mech-mind.net/en/suite-tutorial/latest/vision-system-calibration.html>
- Automatic Calibration, ETH — <https://docs.mech-mind.net/1.7/en-GB/SoftwareSuite/MechVision/Calibration/EthAutoCalib.html>
- Manual Calibration, ETH TCP touch — <https://docs.mech-mind.net/1.7/en-GB/SoftwareSuite/MechVision/Calibration/EthManualCalibTcpTouch.html>
- Calibration Board appendix — <https://docs.mech-mind.net/1.7/en-GB/SoftwareSuite/Appendix/CalibrationBoard/CalibrationBoard.html>

- Calibration Results Check and Analysis — <https://docs.mech-mind.net/1.7/ko-KR/SoftwareSuite/MechVision/Calibration/Reference/CalibResultAnalysis.html>
- Calibration Troubleshooting and FAQs — <https://docs.mech-mind.net/latest/en-GB/SoftwareSuite/MechVision/Calibration/Reference/CalibTroubleshooting.html>

Camera and IPC

- NANO ULTRA-GL specifications — <https://docs.mech-mind.net/en/eye-3d-camera/latest/hardware/specifications-nano-ultra.html>
- NANO ULTRA specifications (2.3.3) — <https://docs.mech-mind.net/en/eye-3d-camera/2.3.3/hardware/specifications-nano-ultra.html>
- Camera models and suitable applications — <https://docs.mech-mind.net/en/eye-3d-camera/2.6.0/hardware/camera-models.html>
- Adjust 3D Exposure Settings — <https://docs.mech-mind.net/en/eye-3d-camera/latest/extended-reading/determine-and-adjust-3d-exposure.html>
- Advanced Parameters for Depth Map and Point Cloud — <https://docs.mech-mind.net/latest/en-GB/MechEye/MechEyeViewer/UsingMechEyeViewer/ParameterAdjustments/DepthMap/DepthMapAdvanced.html>
- Adjust Camera Parameters Using Mech-Eye Viewer — <https://docs.mech-mind.net/en/eye-3d-camera/latest/genicam/adjust-parameters.html>
- IPC Setup — <https://docs.mech-mind.net/latest/en-GB/SoftwareSuite/Appendix/Ipc/IpcConfiguration.html>
- Project docs: mech_mind_camera_md_docs.md, mech_mind_IPC_STD_DOCS.md (attached to this project)

Mech-Viz

- Mech-Viz overview · Fundamentals · Project and Solution — <https://docs.mech-mind.net/1.7/en-GB/SoftwareSuite/MechViz/BasicConcepts/BasicConcepts.html>
- Workflow reference — <https://docs.mech-mind.net/1.7/ko-KR/SoftwareSuite/MechViz/Workflow/Workflow.html>
- Configure the Workflow (pick-place order) — <https://docs.mech-mind.net/1.6/en-GB/SoftwareSuite/MechMindQuickStart/FirstApplication/MechVizSimulatePicking/WorkFlow/WorkFlow.html>
- End Effector · Robot Link · Point Cloud collision configuration — <https://docs.mech-mind.net/1.6/en-GB/SoftwareSuite/MechViz/Collisions/DetectionSettings/EndEffectorSettings/EndEffectorSettings.html>
- Configure Robot Tools and Workobjects — <https://docs.mech-mind.net/1.6/en-GB/SoftwareSuite/MechViz/GettingStarted/ToolsAndWorkobjects/ToolsAndWorkobjects.html>

Application guidance

- Robotic Machine Tending with 3D Vision — <https://www.mech-mind.com/solution/machine-tending.html>

- Vision System Tutorial index — <https://docs.mech-mind.net/en/suite-tutorial/latest/index.html>
 - Vision Solution Design — <https://docs.mech-mind.net/en/suite-tutorial/latest/vision-system-design.html>
 - Workpiece Locating (General Workpiece Recognition) — <https://docs.mech-mind.net/en/suite-tutorial/1.8.3/getting-started/first-application-vision.html>
 - Guide to Improving Application Cycle Time — <https://docs.mech-mind.net/en/suite-tutorial/latest/cycle-time/cycle-time-improvement-guide.html>
 - Error Code Processing — <https://docs.mech-mind.net/en/suite-service-manual/latest/troubleshooting/troubleshooting-common-issues-error-codes.html>
 - Collect Information about the Issue (log paths) — <https://docs.mech-mind.net/1.7/en-GB/SoftwareSuite/Support/Troubleshooting/MustRead/IssueInfoCollection.html>
 - Before You Upgrade — <https://docs.mech-mind.net/1.7/en-GB/SoftwareSuite/ReleaseNote/General.html>
-

End of handbook. Claims marked [?] in the text are collected in Appendix F — verify them on your build before relying on them in a production cell.